ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

д-р техн. наук, доцент		С.И. Колесникова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Модели динамических систем-1

по курсу: КОМПЬЮТЕРНОЕ МОДЕЛИРОВАНИЕ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4232		Н.В. Юрагин
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Цель работ: Цель настоящей работы: освоить приемы моделирования непрерывных процессов в MatLab Simulink.

Вариант №1

1) $y' + 2xy = xe^{-x^2}, \quad y(0) = 2.$

2) $\begin{cases} \dfrac{dx}{dt} = 4x - y \\ \dfrac{dy}{dt} = x + 2y \end{cases}, \quad x(0) = -1, \quad y(0) = 0.$

Ознакомление со справочными сведениями относительно приложения MatLab Simulink:

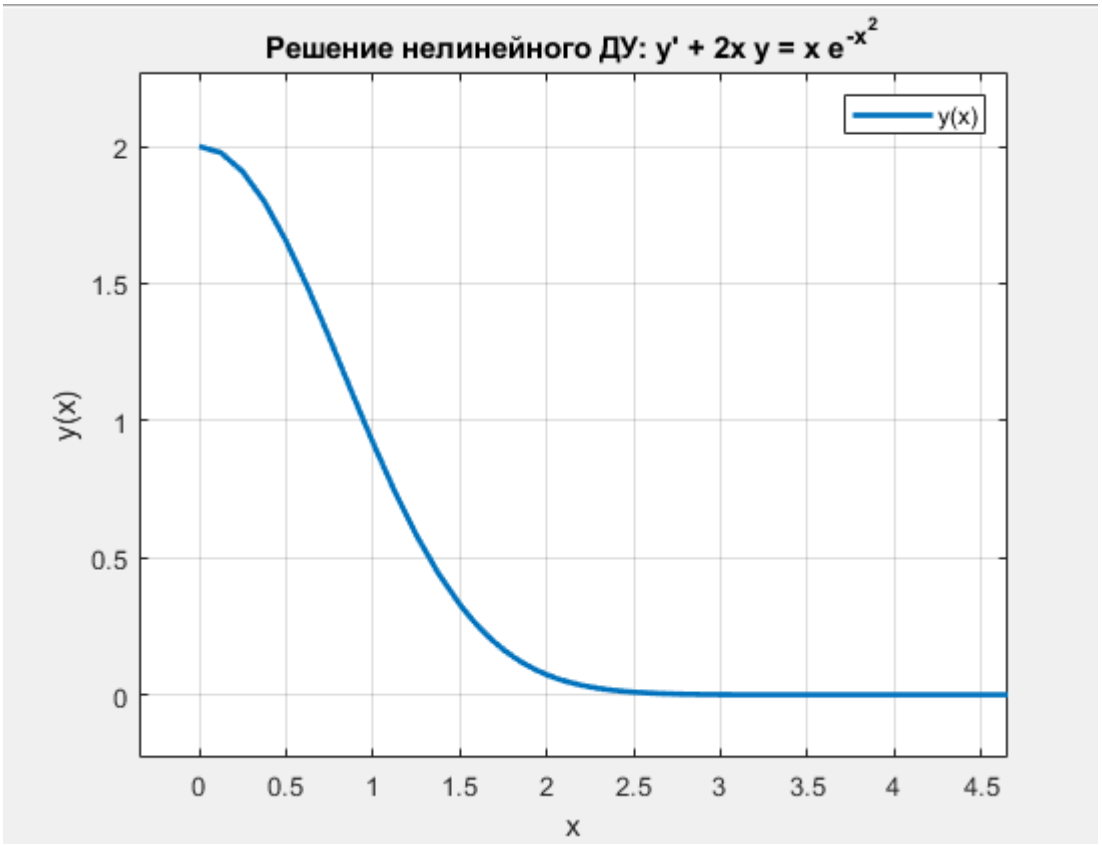
- Simulink – это интерактивное приложение в среде MATLAB, предназначенное для моделирования, анализа и проектирования динамических систем.
- Модели создаются в виде наглядных блок-схем (диаграмм) путем перетаскивания и соединения блоков из обширных библиотек.
- Библиотеки содержат блоки для источников сигналов (Source), приемников (Sink), непрерывных (Continuous) и дискретных (Discrete) систем, математических операций (Math Operations) и др.
- Основные этапы работы: создание модели, настройка параметров блоков и решателя (Solver), запуск моделирования, анализ результатов.

Построение графиков непрерывной нелинейной модели решения ДУ:

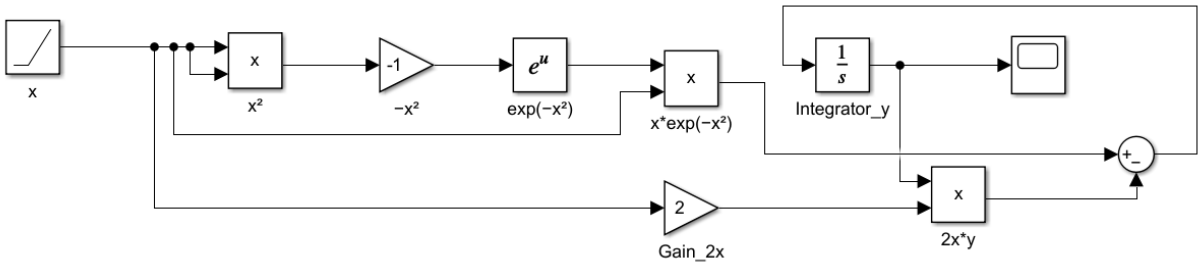
Код:

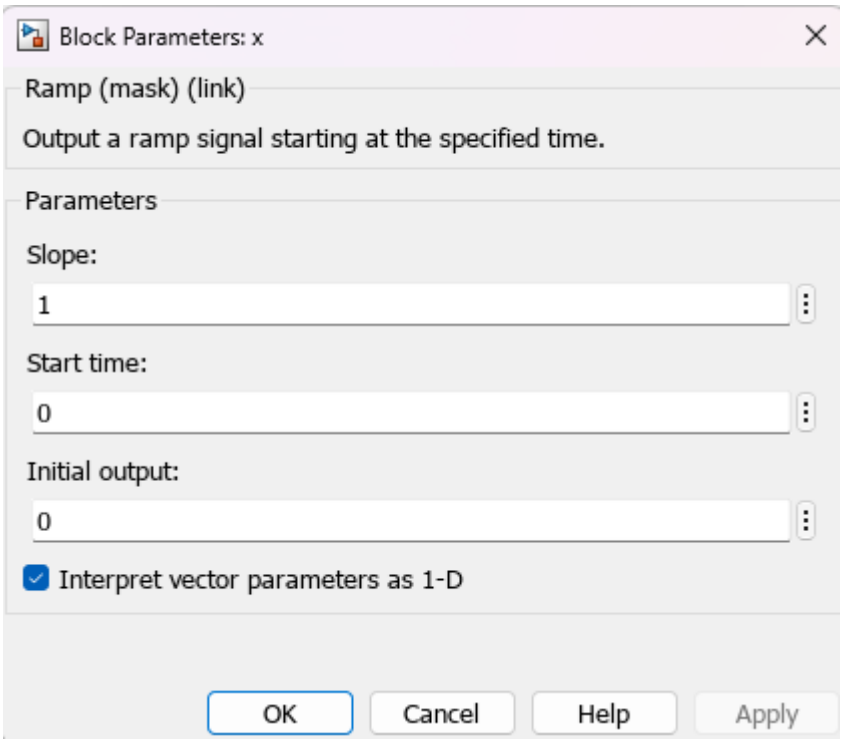
```
% Вариант №1. Решение нелинейного ДУ y' + 2x y = x e^{-x^2}, y(0)=2

% 1. Задаем временной интервал и начальные условия
tspan = [0 5];        % Интервал по x
y0 = 2;                % Начальное условие y(0)=2

% 2. Решаем ОДУ первого порядка с помощью решателя ode45
```

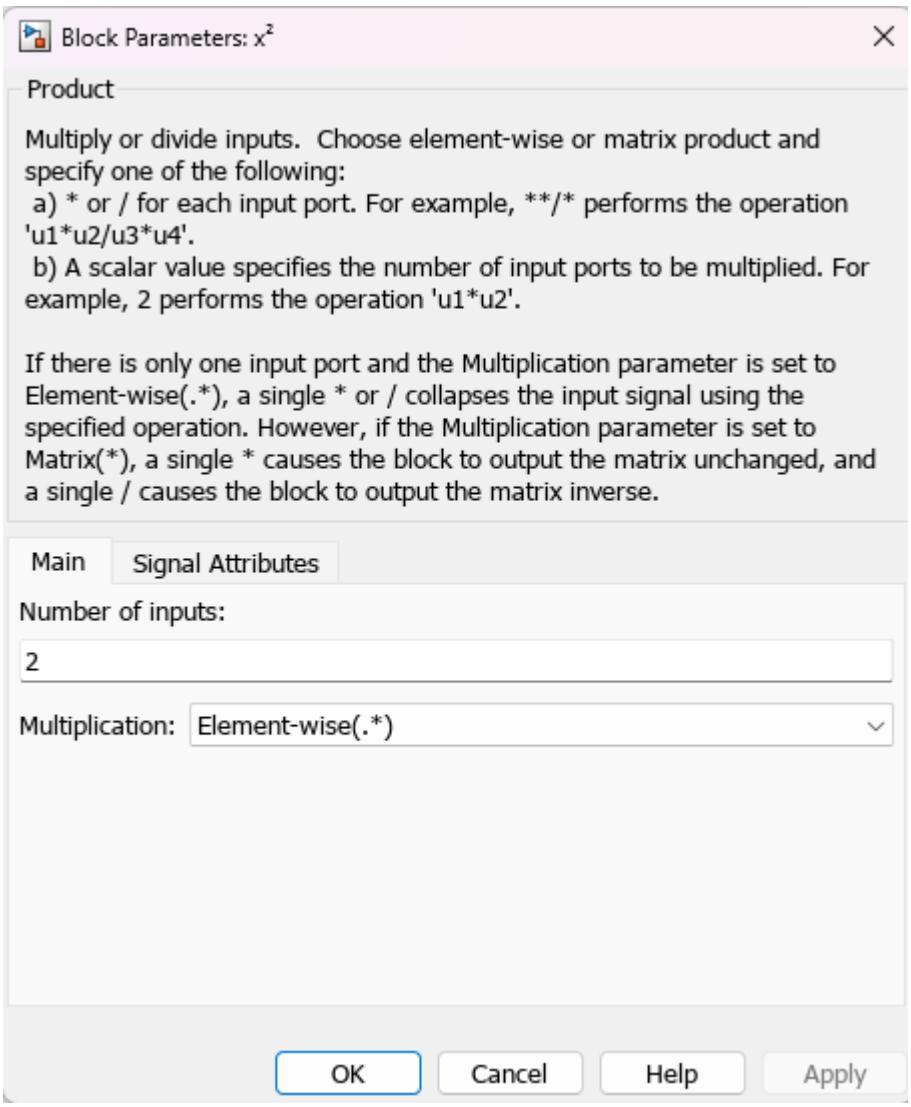
```
[t, y] = ode45(@odefun_nonlin, tspan, y0);

% 3. Строим график
figure;
plot(t, y, 'LineWidth', 2);
grid on;
title('Решение нелинейного ДУ: y'' + 2x y = x e^{-x^2}');
xlabel('x');
ylabel('y(x)');
legend('y(x)');

% 5. Функция, задающая правую часть ОДУ
function dydt = odefun_nonlin(t, y)
    % Здесь y — скаляр (значение функции в точке t=x)
    dydt = t*exp(-t^2) - 2*t*y;   % y' = x e^{-x^2} - 2x y
end
```

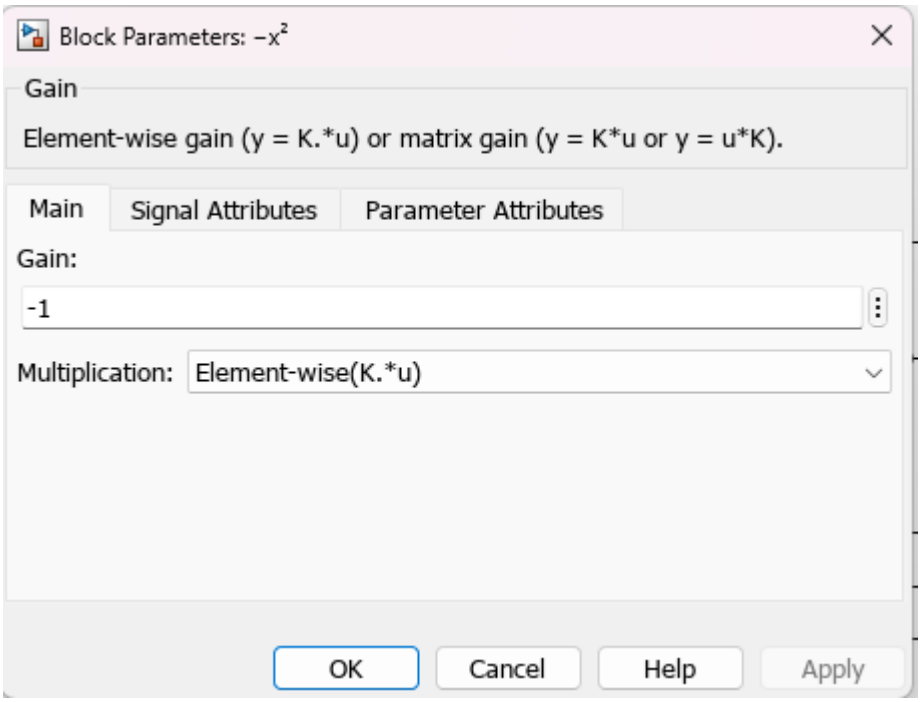


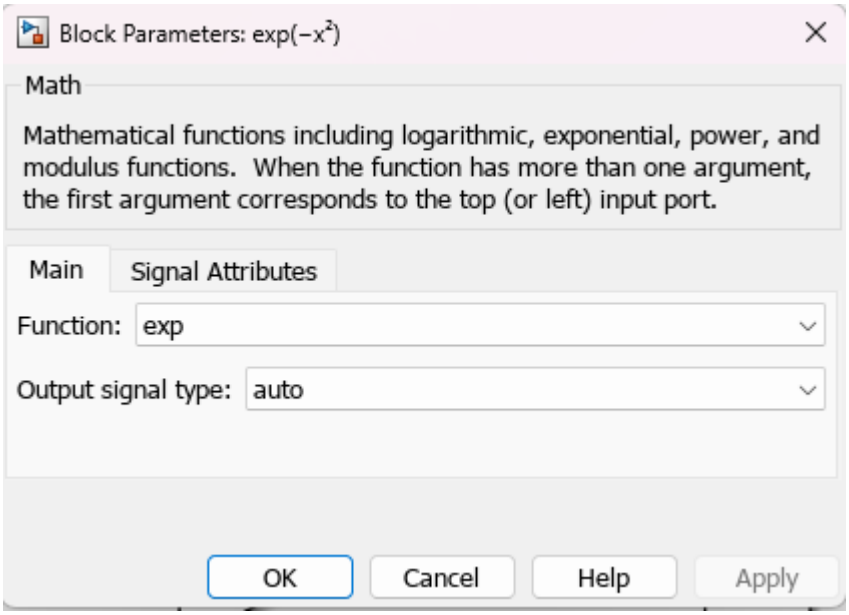
Разработка модели Simulink для решения нелинейного ДУ:

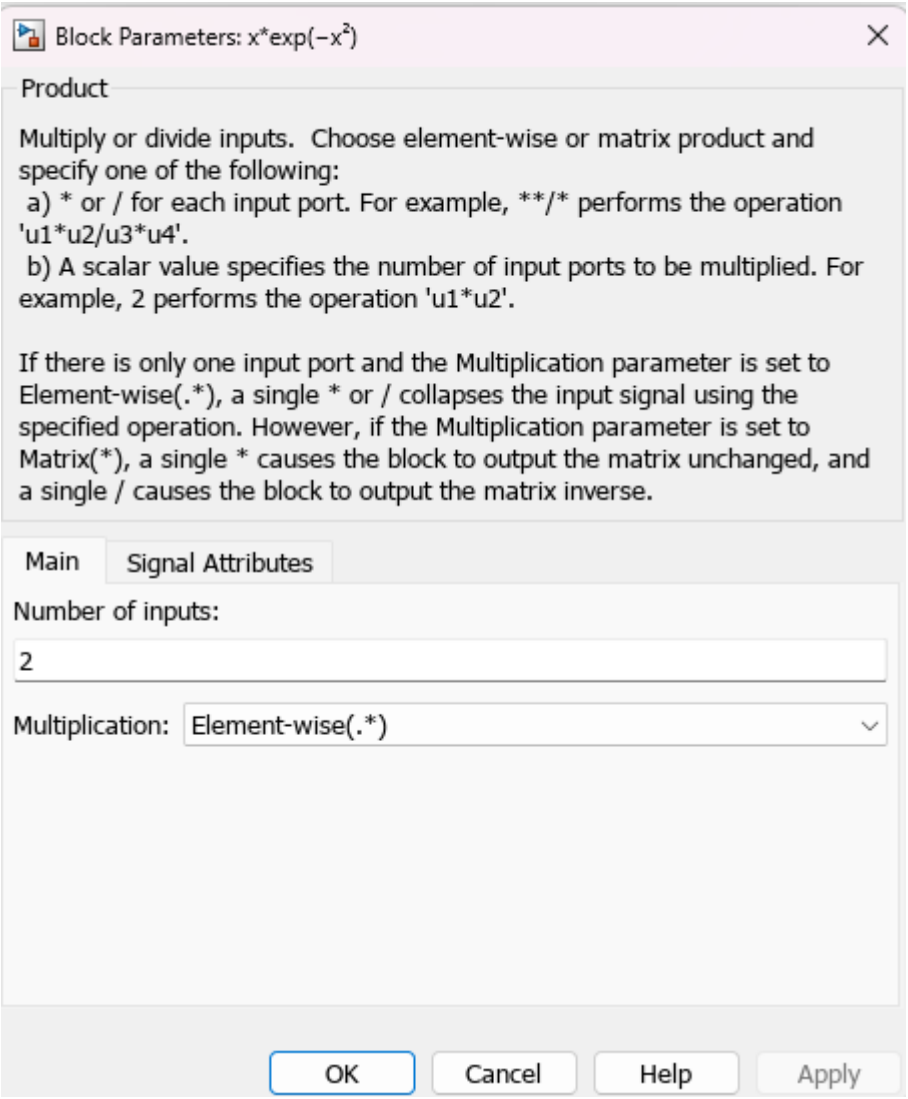





Block Parameters: x²
Product
Multiply or divide inputs. Choose element-wise or matrix product and specify one of the following:
a) * or / for each input port. For example, **/* performs the operation 'u1*u2/u3*u4'.
b) A scalar value specifies the number of input ports to be multiplied. For example, 2 performs the operation 'u1*u2'.
If there is only one input port and the Multiplication parameter is set to Element-wise(.*), a single * or / collapses the input signal using the specified operation. However, if the Multiplication parameter is set to Matrix(*), a single * causes the block to output the matrix unchanged, and a single / causes the block to output the matrix inverse.
Main
Signal Attributes
Number of inputs:
2
Multiplication:
Element-wise(.*)
OK
Cancel
Help
Apply


Block Parameters: −x²
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
-1
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply


Block Parameters: exp(−x²)
Math
Mathematical functions including logarithmic, exponential, power, and modulus functions. When the function has more than one argument, the first argument corresponds to the top (or left) input port.
Main
Signal Attributes
Function:
exp
Output signal type:
auto
OK
Cancel
Help
Apply


Block Parameters: x*exp(−x²)
Product
Multiply or divide inputs. Choose element-wise or matrix product and specify one of the following:
a) * or / for each input port. For example, **/* performs the operation 'u1*u2/u3*u4'.
b) A scalar value specifies the number of input ports to be multiplied. For example, 2 performs the operation 'u1*u2'.
If there is only one input port and the Multiplication parameter is set to Element-wise(.*), a single * or / collapses the input signal using the specified operation. However, if the Multiplication parameter is set to Matrix(*), a single * causes the block to output the matrix unchanged, and a single / causes the block to output the matrix inverse.
Main
Signal Attributes
Number of inputs:
2
Multiplication:
Element-wise(.*)
OK
Cancel
Help
Apply

Block Parameters: Gain_2x

Gain

Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).

Main | Signal Attributes | Parameter Attributes

Gain:

2

Multiplication: Element-wise(K.*u)

OK | Cancel | Help | Apply

Block Parameters: Integrator_y

Integrator

Continuous-time integration of the input signal.

Parameters

External reset: none

Initial condition source: internal

Initial condition:

2

- [ ] Limit output
- [ ] Wrap state
- [ ] Show saturation port
- [ ] Show state port

Absolute tolerance:

auto

- [ ] Ignore limit and reset when linearizing
- [x] Enable zero-crossing detection

State Name: (e.g., 'position')

''

OK | Cancel | Help | Apply

Block Parameters: 2x*y

Product

Multiply or divide inputs. Choose element-wise or matrix product and specify one of the following:
a) * or / for each input port. For example, **/* performs the operation 'u1*u2/u3*u4'.
b) A scalar value specifies the number of input ports to be multiplied. For example, 2 performs the operation 'u1*u2'.

If there is only one input port and the Multiplication parameter is set to Element-wise(.*), a single * or / collapses the input signal using the specified operation. However, if the Multiplication parameter is set to Matrix(*), a single * causes the block to output the matrix unchanged, and a single / causes the block to output the matrix inverse.

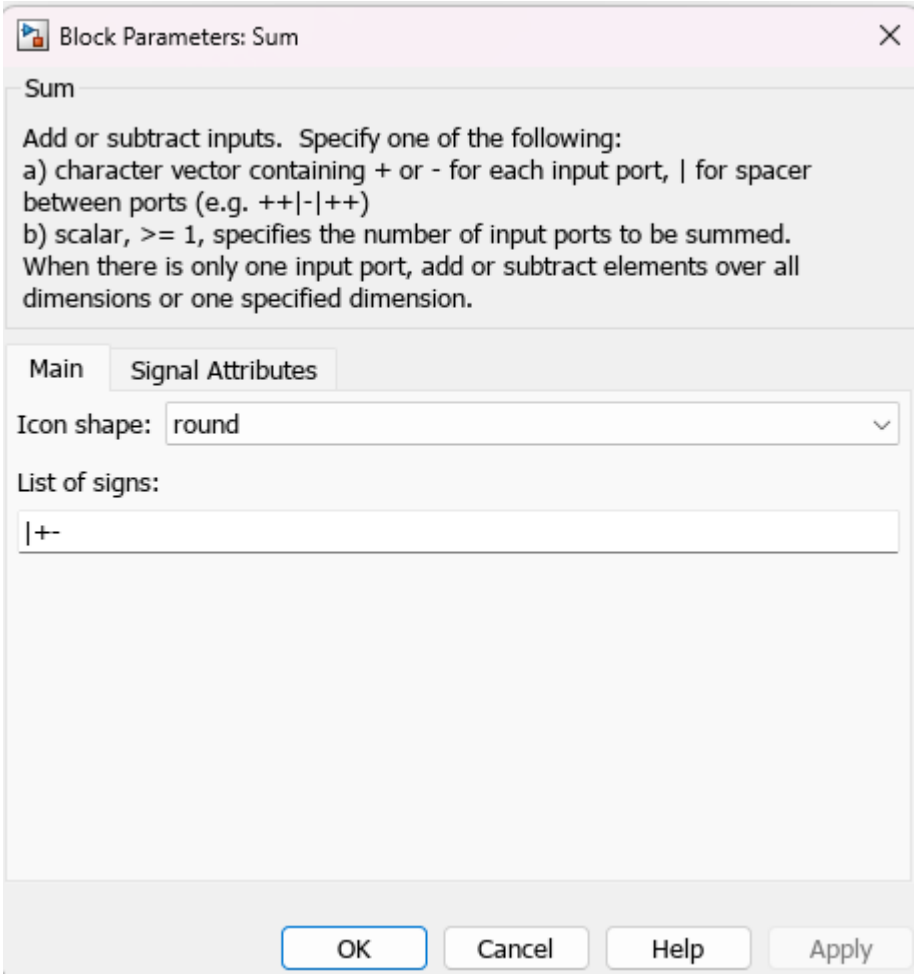
Main | Signal Attributes

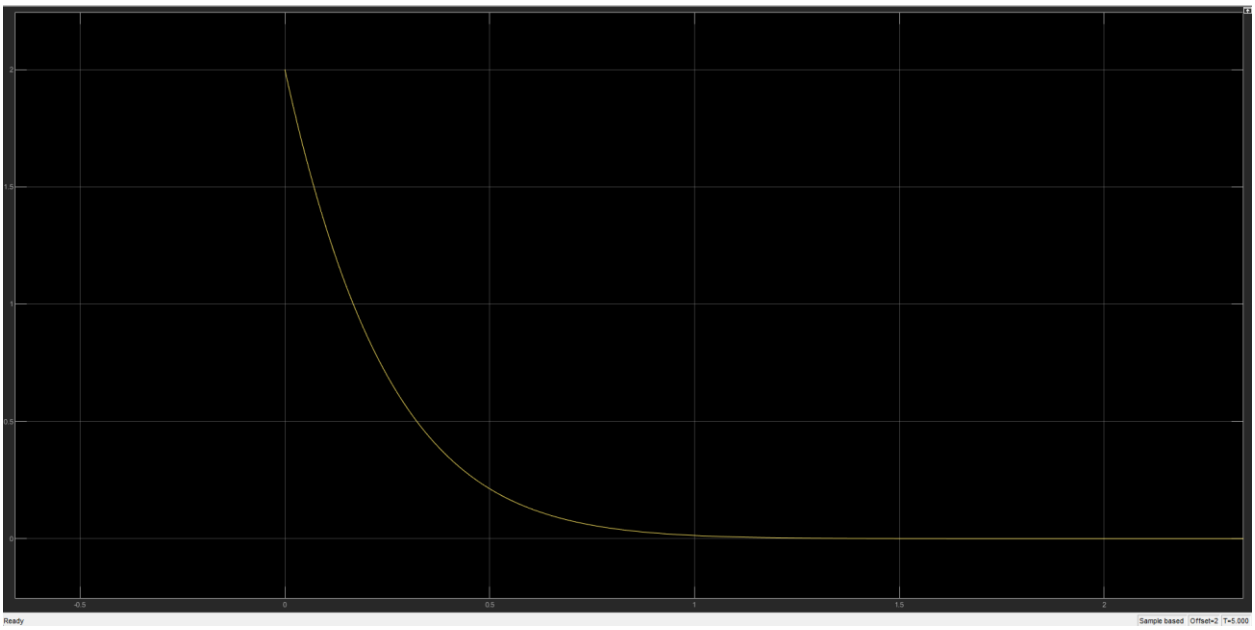
Number of inputs:

2

Multiplication: Element-wise(.*)

OK | Cancel | Help | Apply

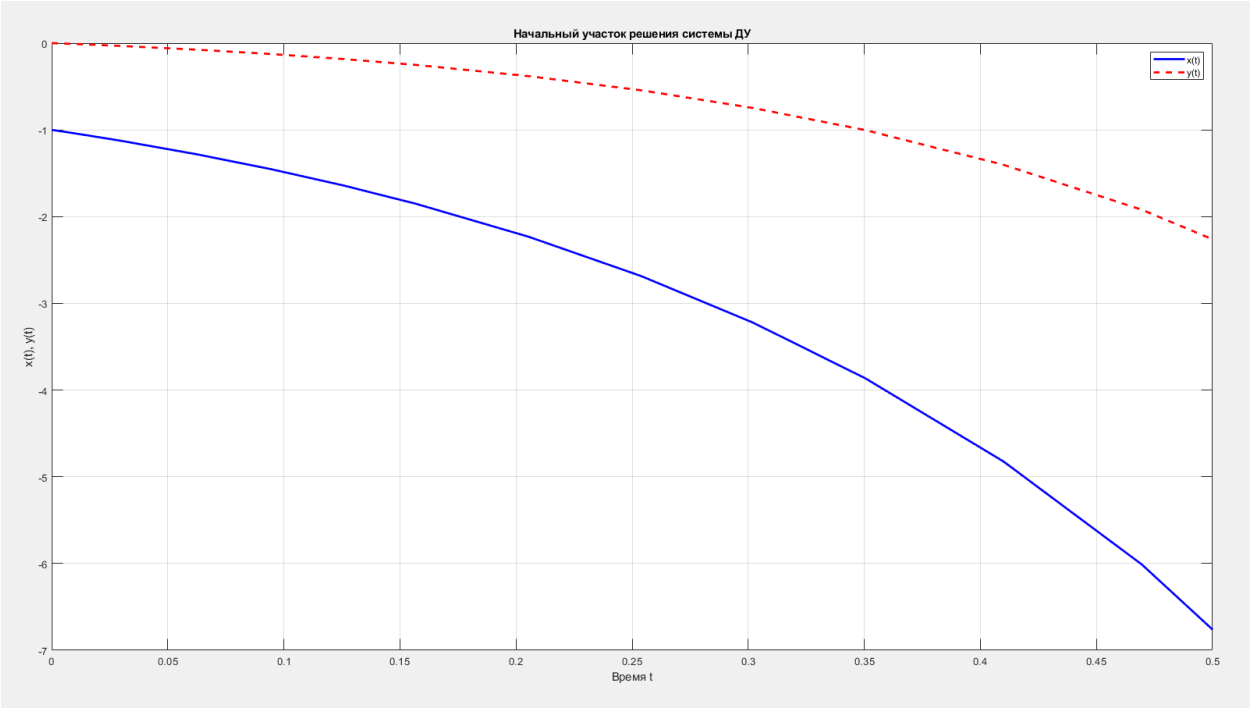

Block Parameters: Sum
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
round
List of signs:
|+-
OK
Cancel
Help
Apply

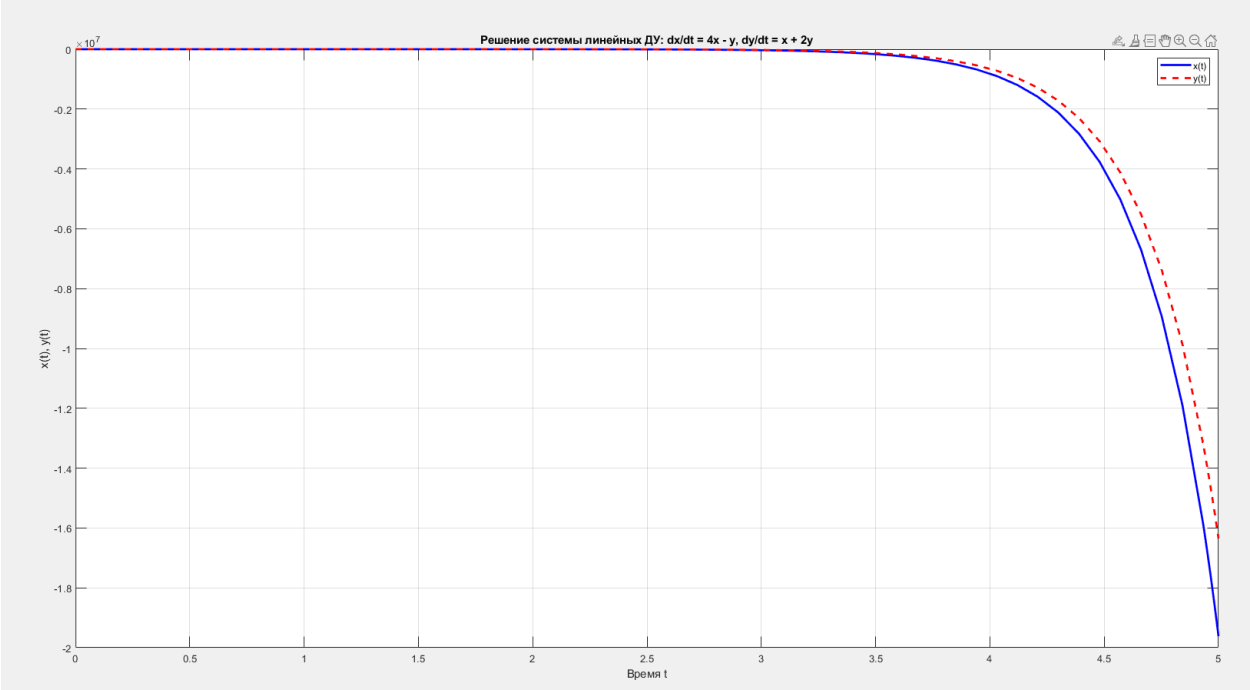


Построение графиков непрерывной линейной модели решения системы ДУ:

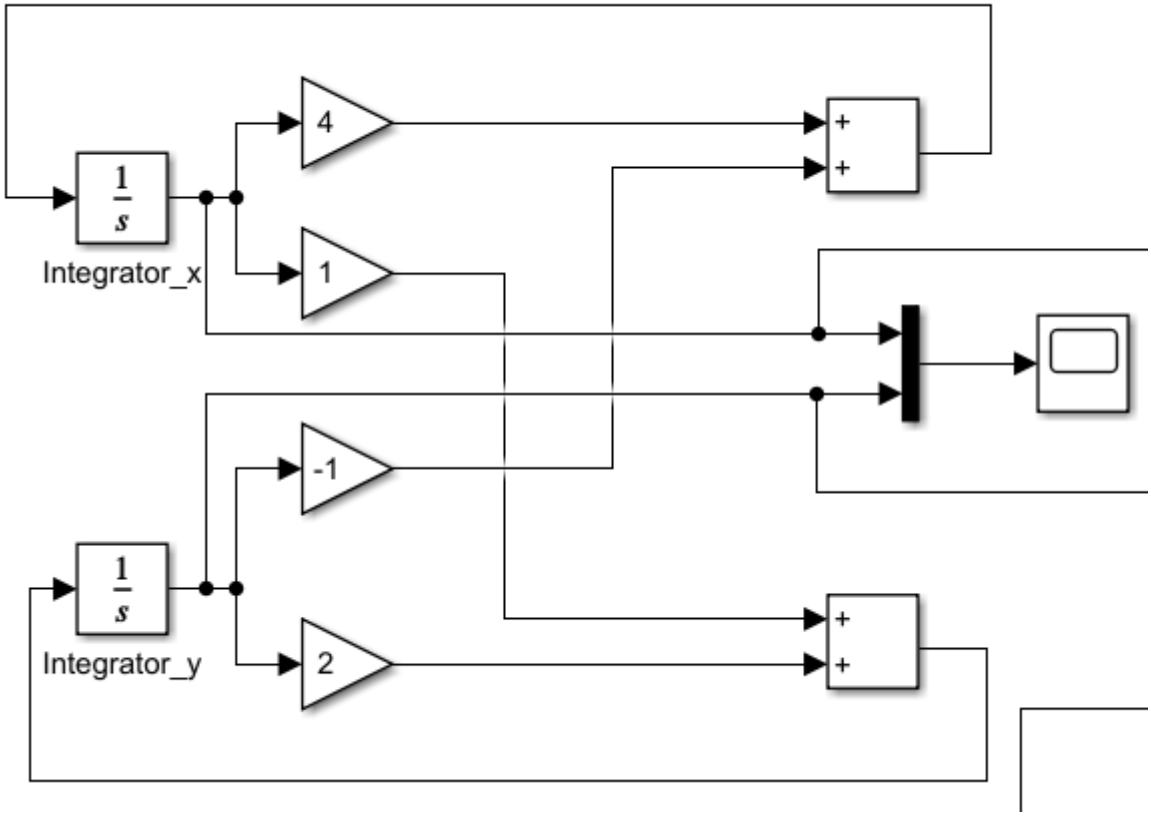
Код:

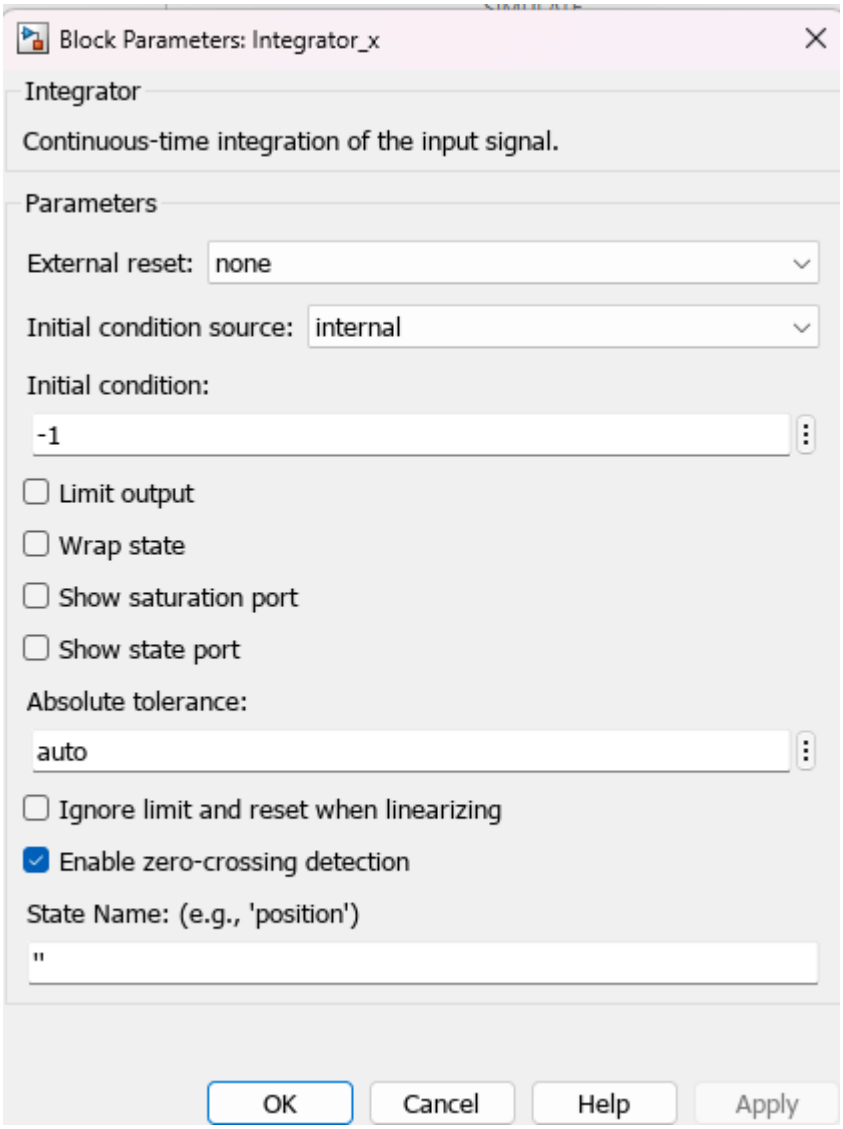
```
% Пункт 2. Решение системы линейных ДУ
% dx/dt = 4x - y
% dy/dt = x + 2y
% с начальными условиями x(0)=-1, y(0)=0

% 1. Задаем матрицу коэффициентов A
A = [4  -1;
     1   2];

% 2. Задаем временной интервал и начальные условия
tspan = [0 5];          % Время от 0 до 5
init_cond = [-1; 0];    % Вектор [x(0); y(0)]

% 3. Решаем систему ОДУ с помощью решателя ode45
[t, XY] = ode45(@(t, xy) A * xy, tspan, init_cond);

% 4. Извлекаем решения x(t) и y(t)
x = XY(:, 1);
y = XY(:, 2);

% 5. Строим графики
figure;
plot(t, x, 'b-', 'LineWidth', 2);
hold on;
plot(t, y, 'r--', 'LineWidth', 2);
grid on;
title('Решение системы линейных ДУ: dx/dt = 4x - y, dy/dt = x + 2y');
xlabel('Время t');
ylabel('x(t), y(t)');
legend('x(t)', 'y(t)');
hold off;

% 6. Дополнительный график для крупного плана в начале
figure;
plot(t, x, 'b-', t, y, 'r--', 'LineWidth', 2);
xlim([0 0.5]);          % Смотрим на начало процесса
grid on;
title('Начальный участок решения системы ДУ');
xlabel('Время t');
ylabel('x(t), y(t)');
legend('x(t)', 'y(t)');
```

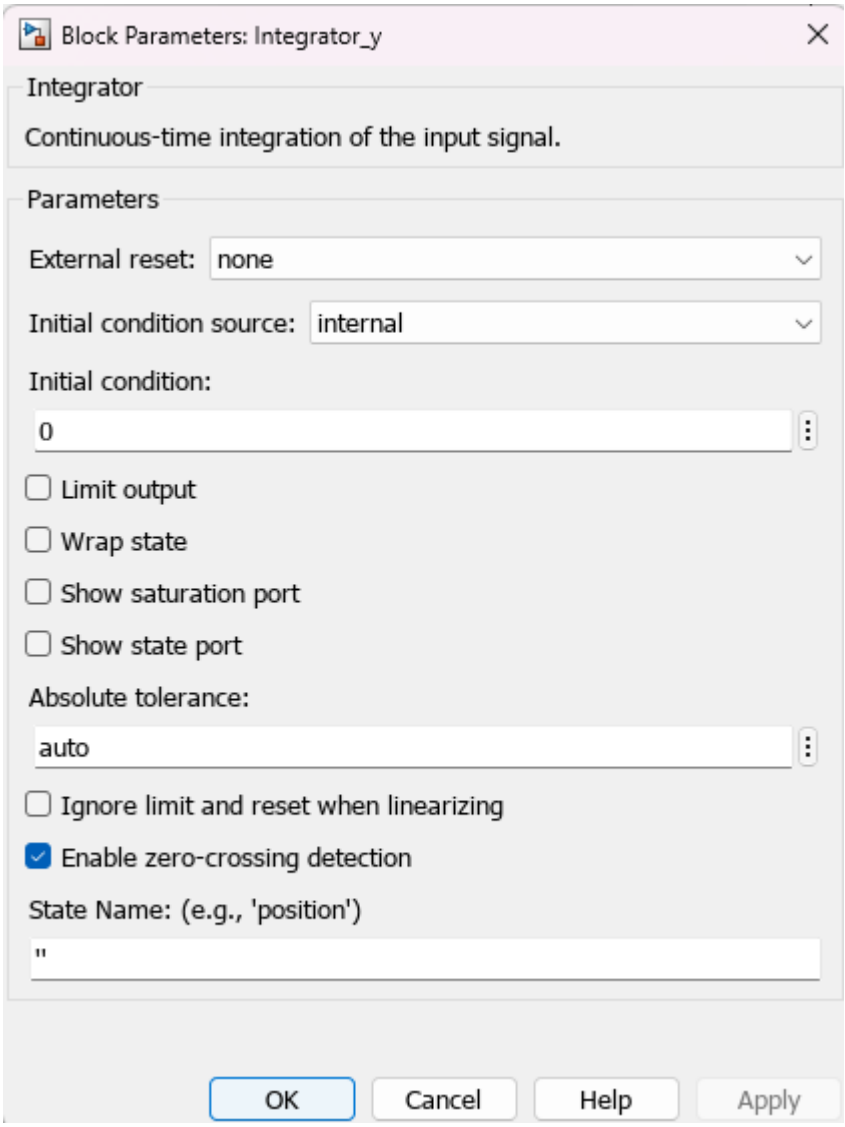

Начальный участок решения системы ДУ
x(t)
y(t)
x(t), y(t)
Время t
0
-1
-2
-3
-4
-5
-6
-7
0
0.05
0.1
0.15
0.2
0.25
0.3
0.35
0.4
0.45
0.5

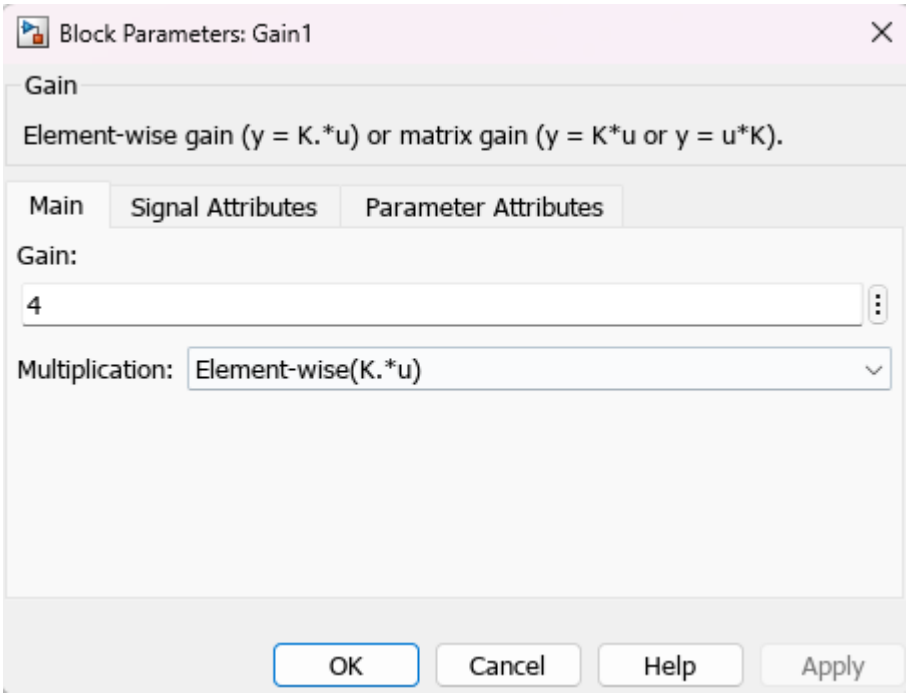

Решение системы линейных ДУ: dx/dt = 4x - y, dy/dt = x + 2y
×10^7
x(t)
y(t)
x(t), y(t)
Время t
0
-0.2
-0.4
-0.6
-0.8
-1
-1.2
-1.4
-1.6
-1.8
-2
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5

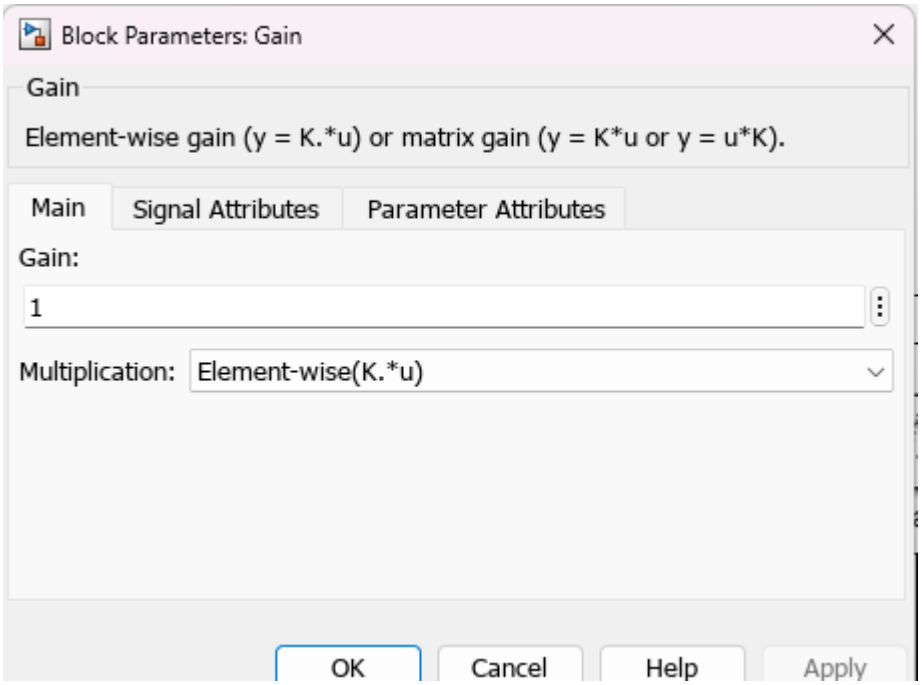
Разработка моделей Simulink для решения системы ДУ:

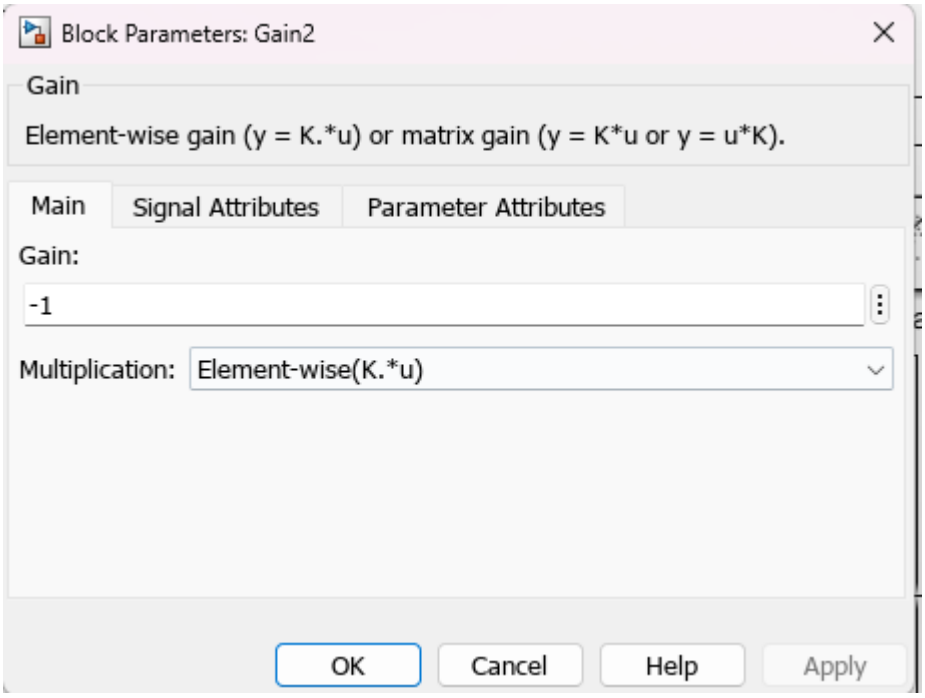


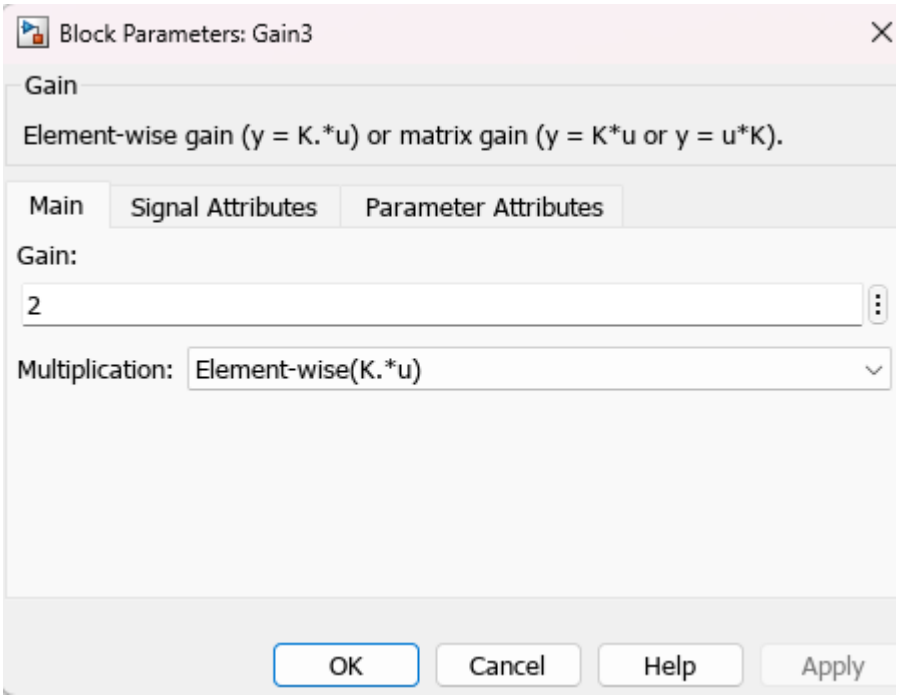

Block Parameters: Integrator_x
Integrator
Continuous-time integration of the input signal.
Parameters
External reset:
none
Initial condition source:
internal
Initial condition:
-1
Limit output
Wrap state
Show saturation port
Show state port
Absolute tolerance:
auto
Ignore limit and reset when linearizing
Enable zero-crossing detection
State Name: (e.g., 'position')
''
OK
Cancel
Help
Apply

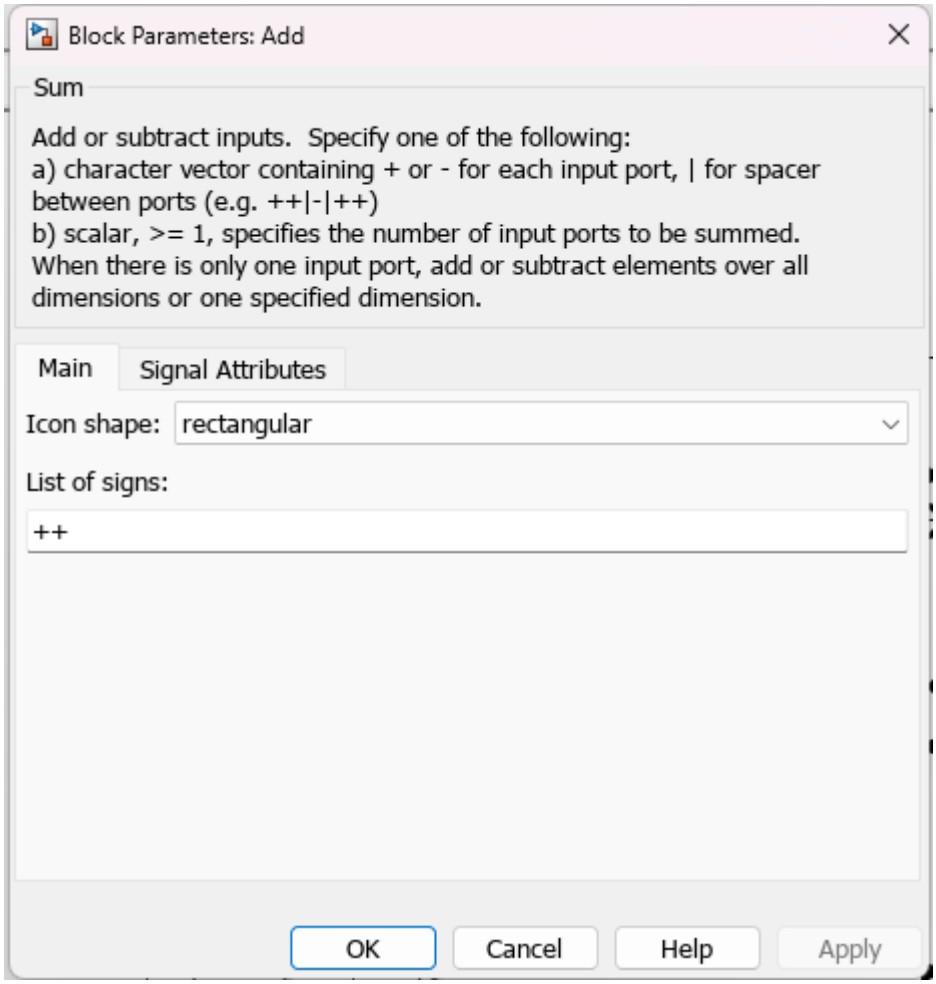

Block Parameters: Integrator_y
Integrator
Continuous-time integration of the input signal.
Parameters
External reset: none
Initial condition source: internal
Initial condition:
0
Limit output
Wrap state
Show saturation port
Show state port
Absolute tolerance:
auto
Ignore limit and reset when linearizing
Enable zero-crossing detection
State Name: (e.g., 'position')
''
OK
Cancel
Help
Apply

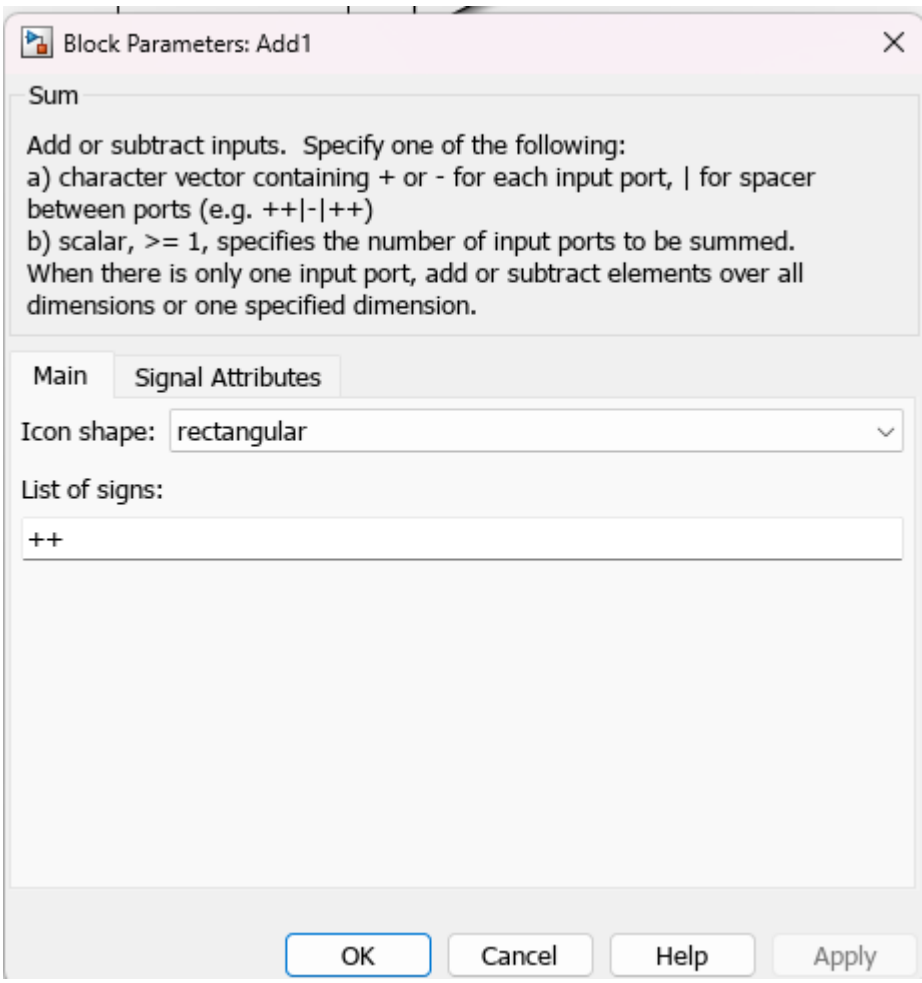

Block Parameters: Gain1
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
4
Multiplication: Element-wise(K.*u)
OK
Cancel
Help
Apply

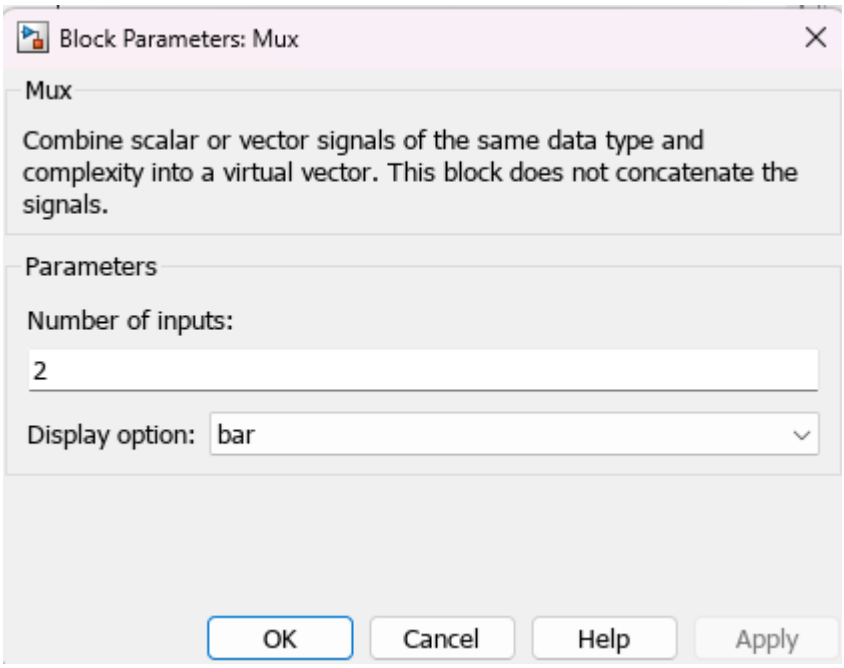

Block Parameters: Gain
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
1
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

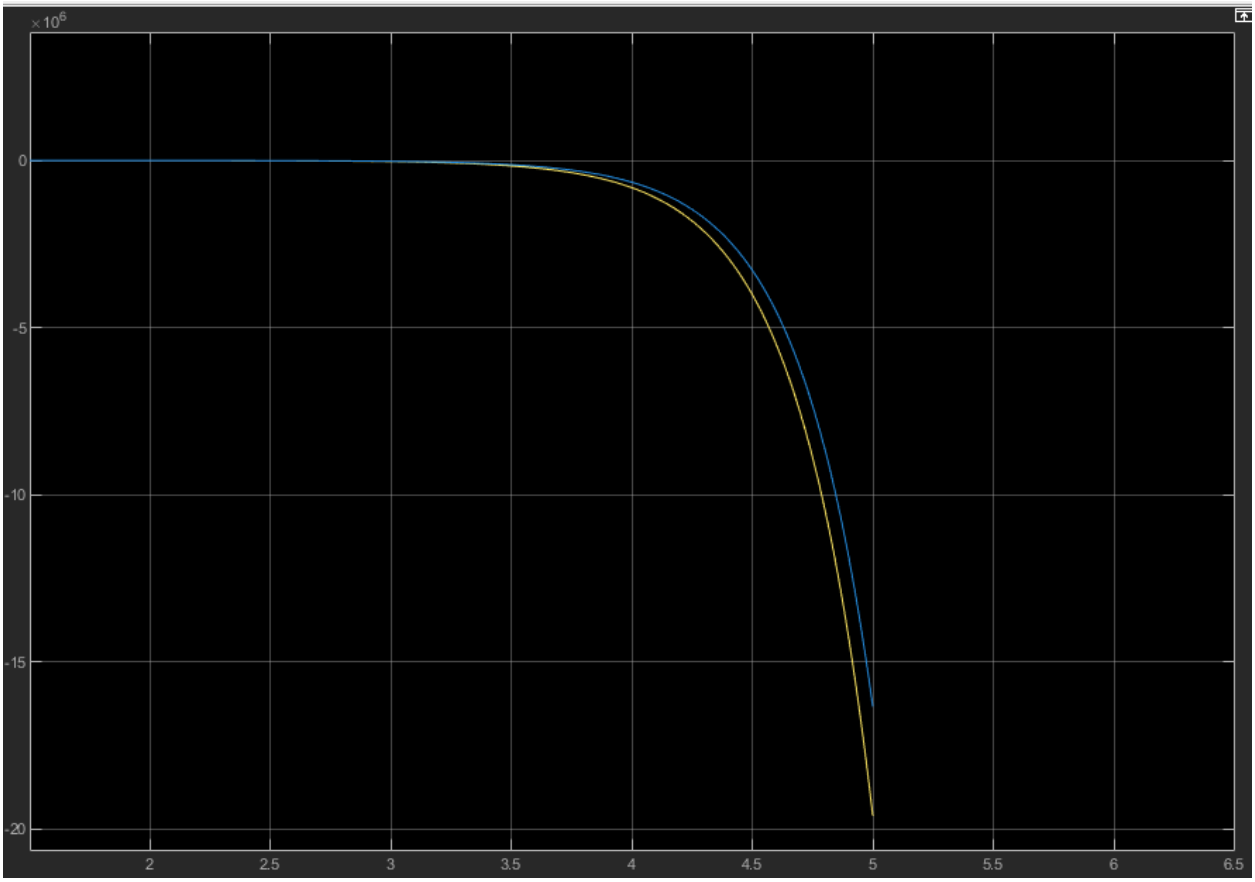

Block Parameters: Gain2
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
-1
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

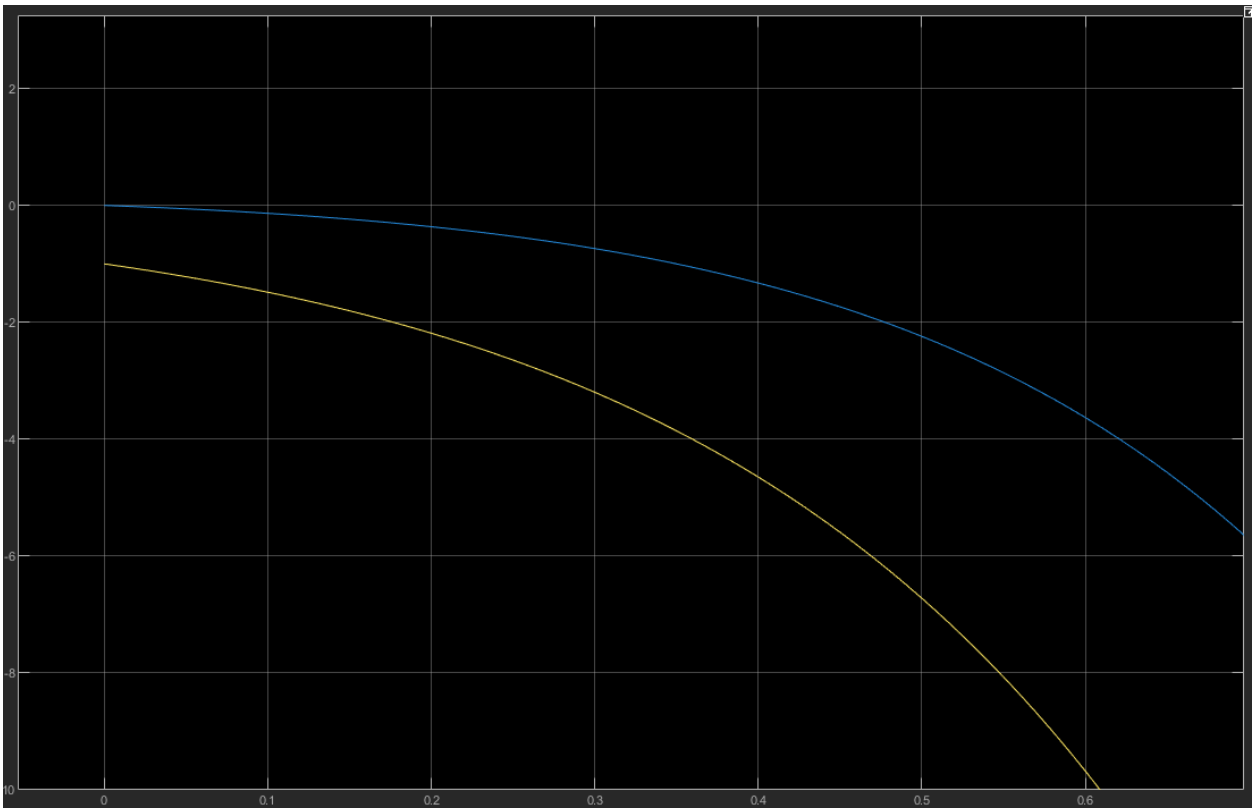

Block Parameters: Gain3
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
2
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply


Block Parameters: Add
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed. When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
rectangular
List of signs:
++
OK
Cancel
Help
Apply


Block Parameters: Add1
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
rectangular
List of signs:
++
OK
Cancel
Help
Apply


Block Parameters: Mux
Mux
Combine scalar or vector signals of the same data type and complexity into a virtual vector. This block does not concatenate the signals.
Parameters
Number of inputs:
2
Display option:
bar
OK
Cancel
Help
Apply

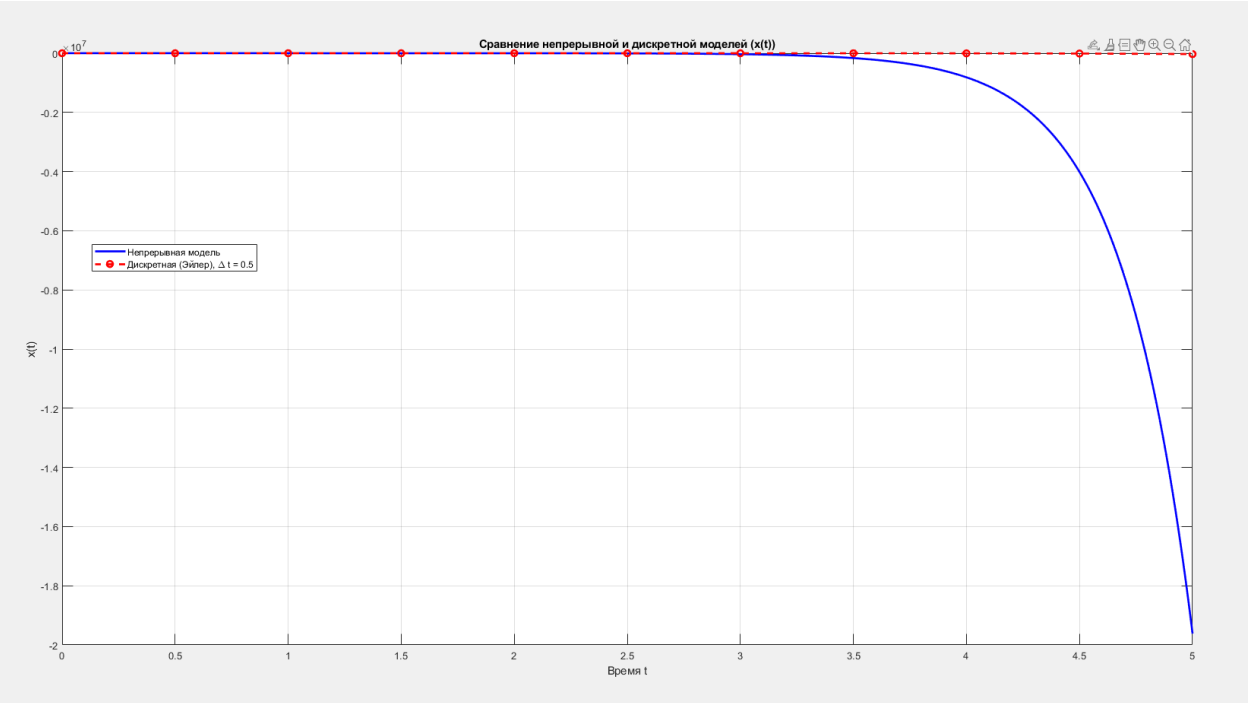

×10^6
0
-5
-10
-15
-20
2
2.5
3
3.5
4
4.5
5
5.5
6
6.5

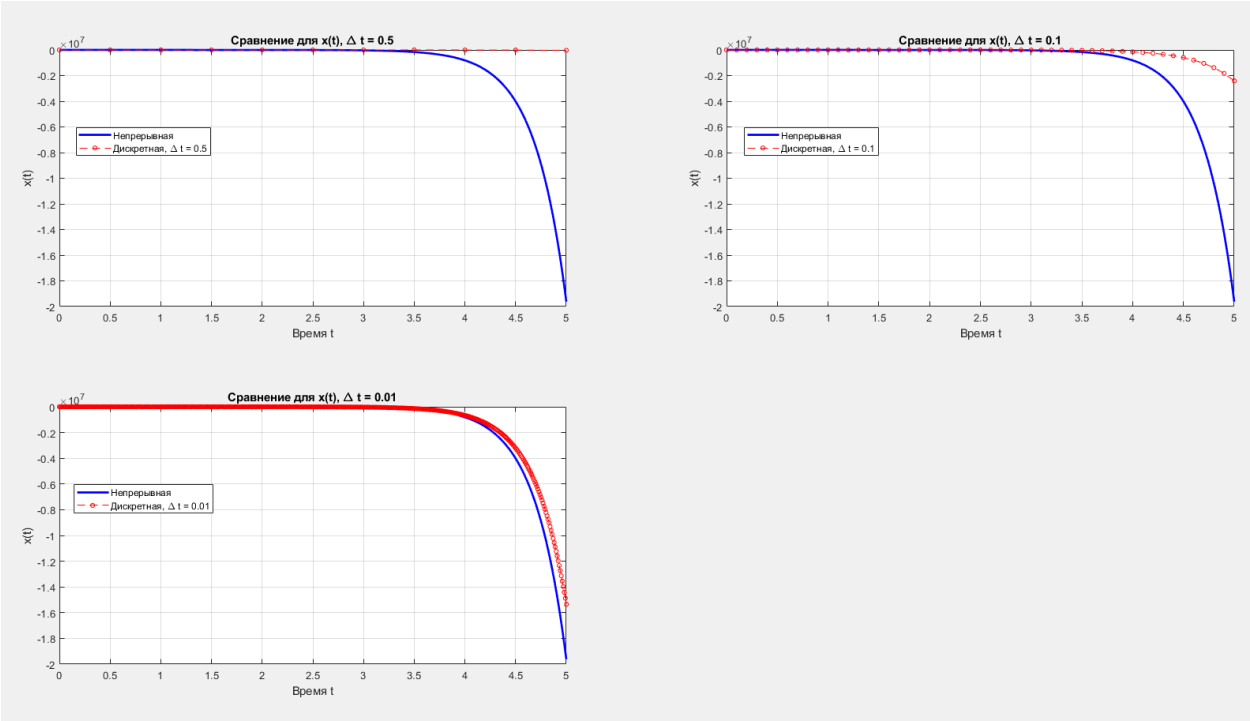

2
0
-2
-4
-6
-8
-10
0
0.1
0.2
0.3
0.4
0.5
0.6

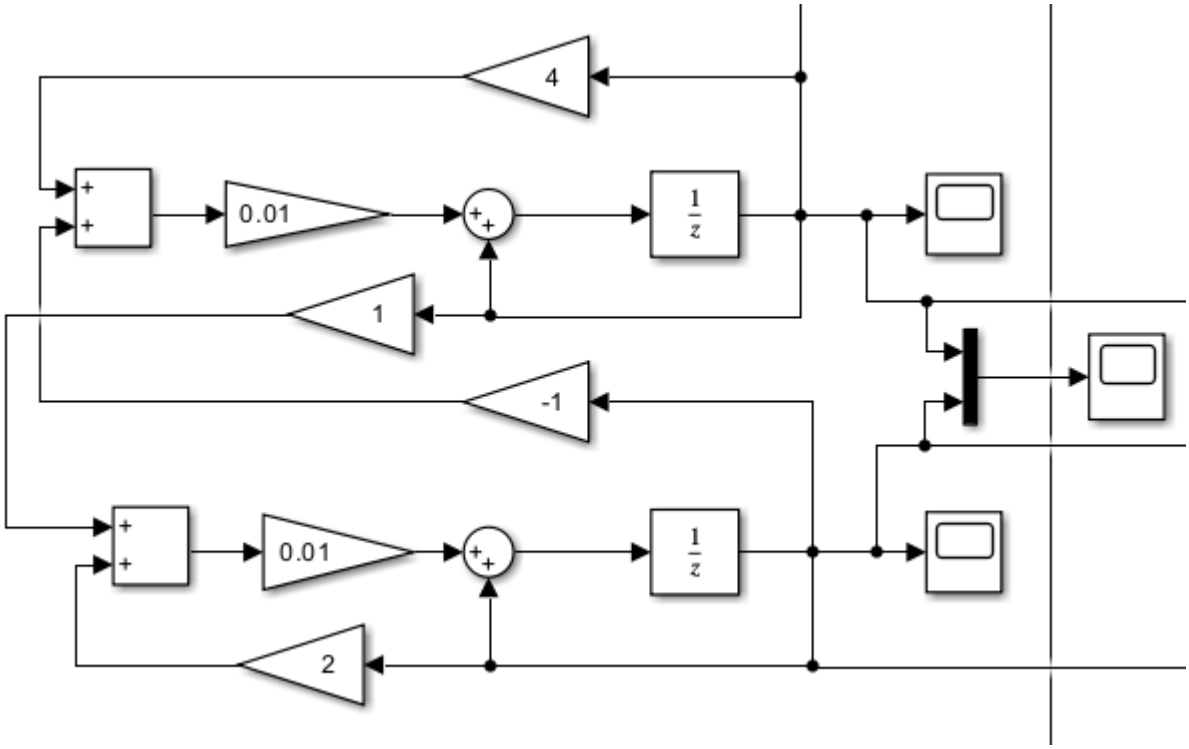
Дискретизация системы и сравнительный анализ:

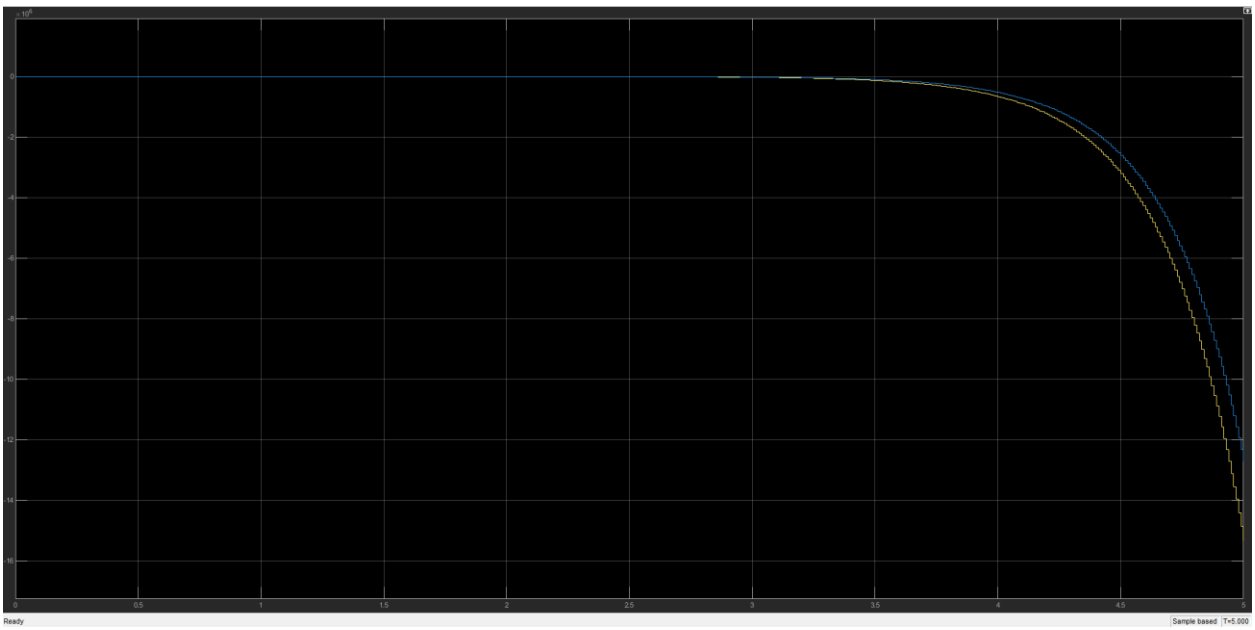
```
% Пункт 6. Сравнение непрерывной и дискретной моделей
% Система:
% dx/dt = 4x - y
% dy/dt = x + 2y
% x(0) = -1,  y(0) = 0

% Параметры системы
A = [4  -1;
     1   2];
init_cond = [-1; 0];
tspan = [0 5];
t_continuous = linspace(tspan(1), tspan(2), 1000); % Вектор времени для точного
решения

% 1. Точное решение (численное, через ode45)
[~, XY_cont] = ode45(@(t, xy) A * xy, t_continuous, init_cond);
x_cont = XY_cont(:, 1);
y_cont = XY_cont(:, 2);

% 2. Дискретное решение методом Эйлера с разными шагами
dt_values = [0.5, 0.1, 0.01]; % Шаги дискретизации
figure;
for i = 1:length(dt_values)
    dt = dt_values(i);
    t_discrete = tspan(1):dt:tspan(2); % Вектор времени для дискретного решения

    % Инициализация
    x_disc = zeros(size(t_discrete));
    y_disc = zeros(size(t_discrete));
    x_disc(1) = init_cond(1);
    y_disc(1) = init_cond(2);

    % Метод Эйлера
    for k = 1:length(t_discrete)-1
        dx_dt = 4*x_disc(k) - y_disc(k);
        dy_dt = x_disc(k) + 2*y_disc(k);
        x_disc(k+1) = x_disc(k) + dt * dx_dt;
        y_disc(k+1) = y_disc(k) + dt * dy_dt;
    end

    % Строим графики для x(t)
    subplot(2, 2, i);
    plot(t_continuous, x_cont, 'b-', 'LineWidth', 2, 'DisplayName',
'Непрерывная');
    hold on;
    plot(t_discrete, x_disc, 'ro--', 'MarkerSize', 4, 'DisplayName',
['Дискретная, \Delta t = ', num2str(dt)]);
    hold off;
    grid on;
    title(['Сравнение для x(t), \Delta t = ', num2str(dt)]);
    xlabel('Время t');
    ylabel('x(t)');
    legend('Location','best');
end

% 3. Отдельный график для демонстрации поведения при большом dt (неустойчивость
или расхождение)
dt_bad = 0.5;
t_bad = tspan(1):dt_bad:tspan(2);
x_bad = zeros(size(t_bad));
```

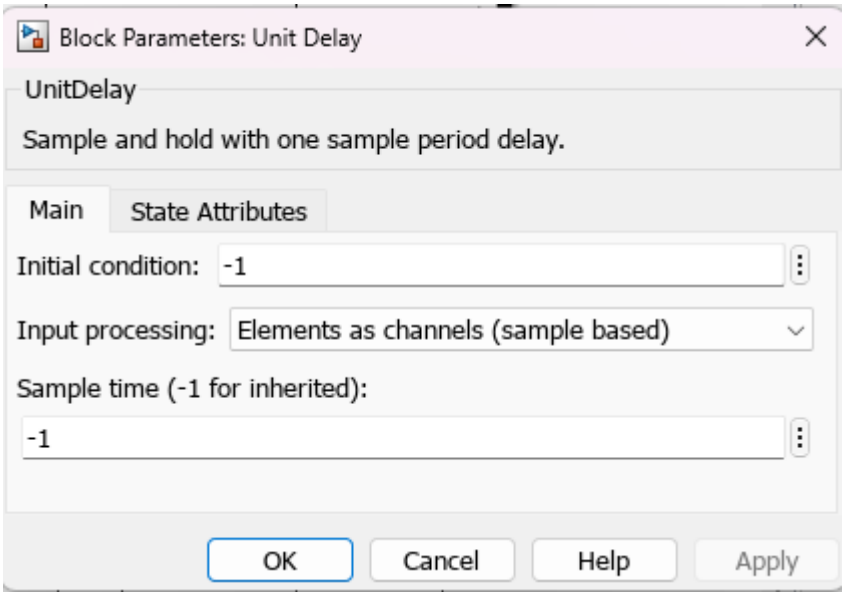
```
y_bad = zeros(size(t_bad));
x_bad(1) = init_cond(1);
y_bad(1) = init_cond(2);

for k = 1:length(t_bad)-1
    dx_dt = 4*x_bad(k) - y_bad(k);
    dy_dt = x_bad(k) + 2*y_bad(k);
    x_bad(k+1) = x_bad(k) + dt_bad * dx_dt;
    y_bad(k+1) = y_bad(k) + dt_bad * dy_dt;
end

figure;
plot(t_continuous, x_cont, 'b-', 'LineWidth', 2, 'DisplayName', 'Непрерывная
модель');
hold on;
plot(t_bad, x_bad, 'ro--', 'LineWidth', 2, 'MarkerSize', 6, 'DisplayName',
['Дискретная (Эйлер), \Delta t = ', num2str(dt_bad)]);
hold off;
grid on;
title('Сравнение непрерывной и дискретной моделей (x(t))');
xlabel('Время t');
ylabel('x(t)');
legend('Location','best');
```

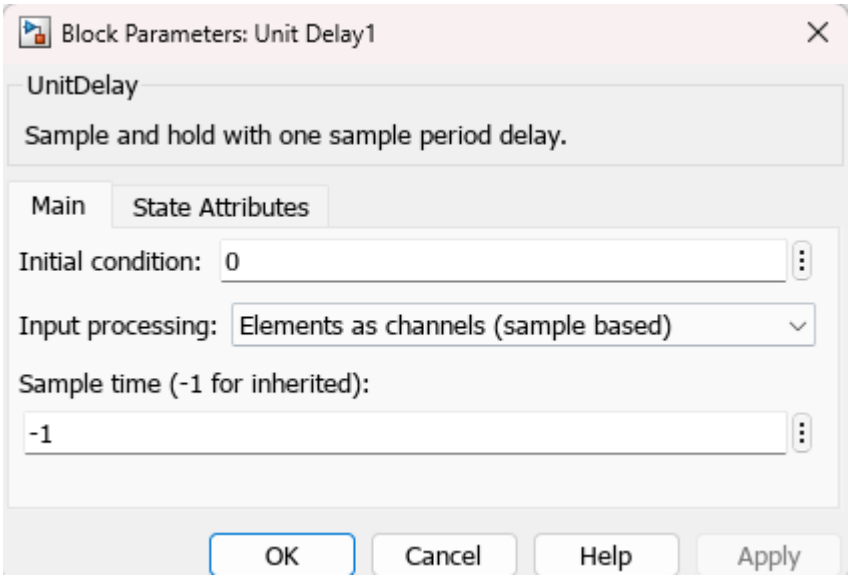


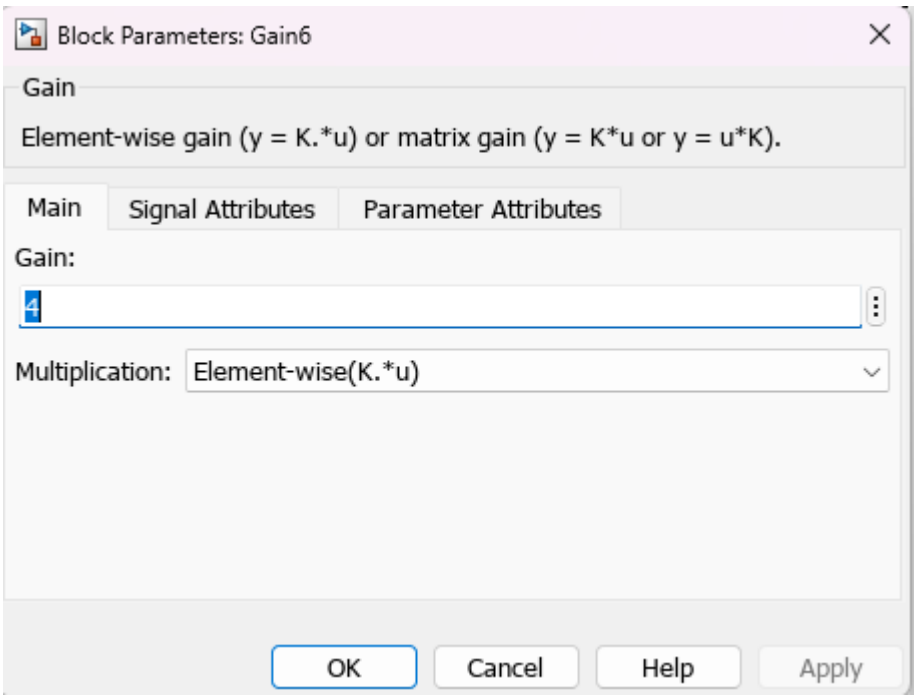

Сравнение для x(t), Δ t = 0.5
Непрерывная
Дискретная, Δ t = 0.5
x(t)
Время t
Сравнение для x(t), Δ t = 0.1
Непрерывная
Дискретная, Δ t = 0.1
x(t)
Время t
Сравнение для x(t), Δ t = 0.01
Непрерывная
Дискретная, Δ t = 0.01
x(t)
Время t

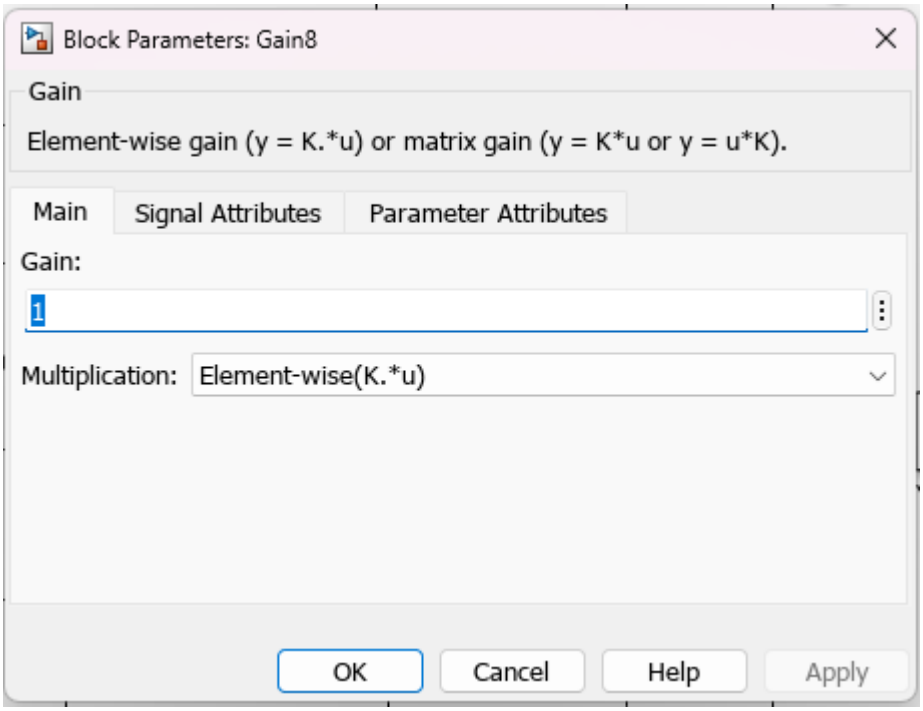

4
0.01
1
-1
0.01
2

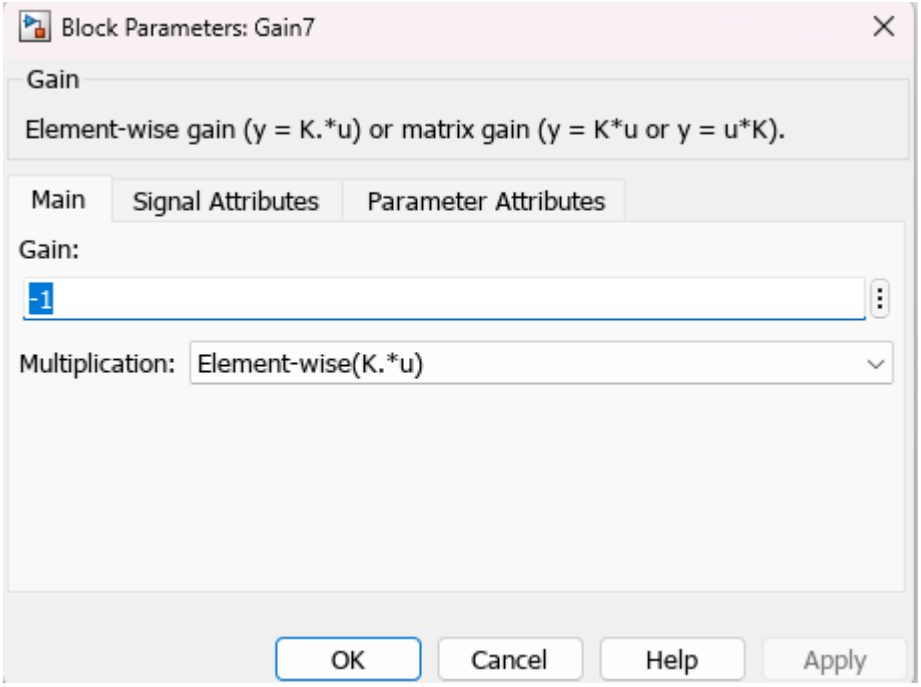


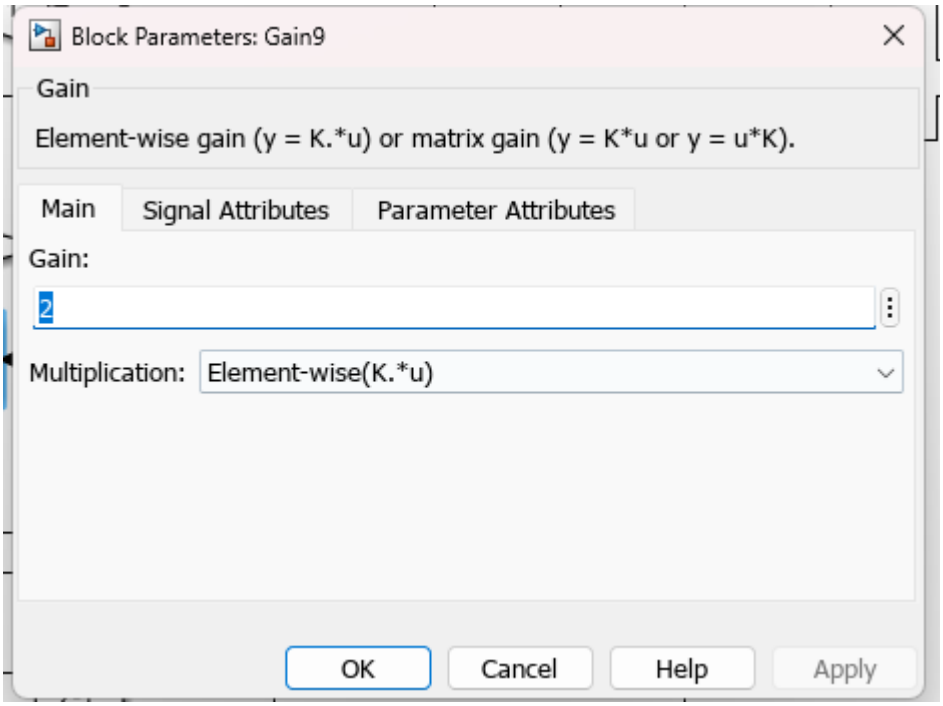

Block Parameters: Unit Delay
UnitDelay
Sample and hold with one sample period delay.
Main
State Attributes
Initial condition:
-1
Input processing:
Elements as channels (sample based)
Sample time (-1 for inherited):
-1
OK
Cancel
Help
Apply

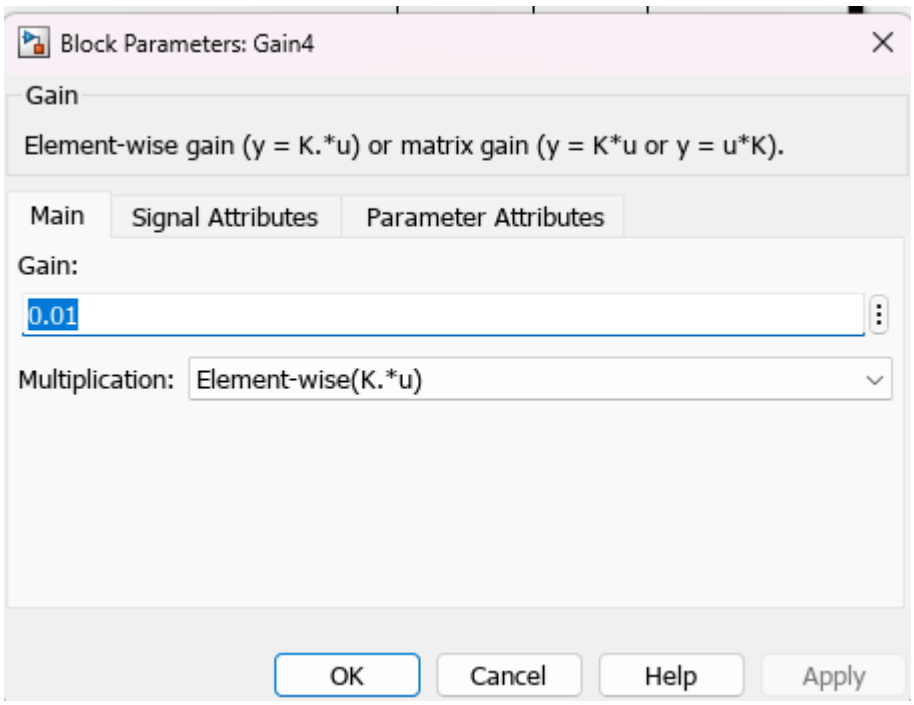

Block Parameters: Unit Delay1
UnitDelay
Sample and hold with one sample period delay.
Main
State Attributes
Initial condition:
0
Input processing:
Elements as channels (sample based)
Sample time (-1 for inherited):
-1
OK
Cancel
Help
Apply

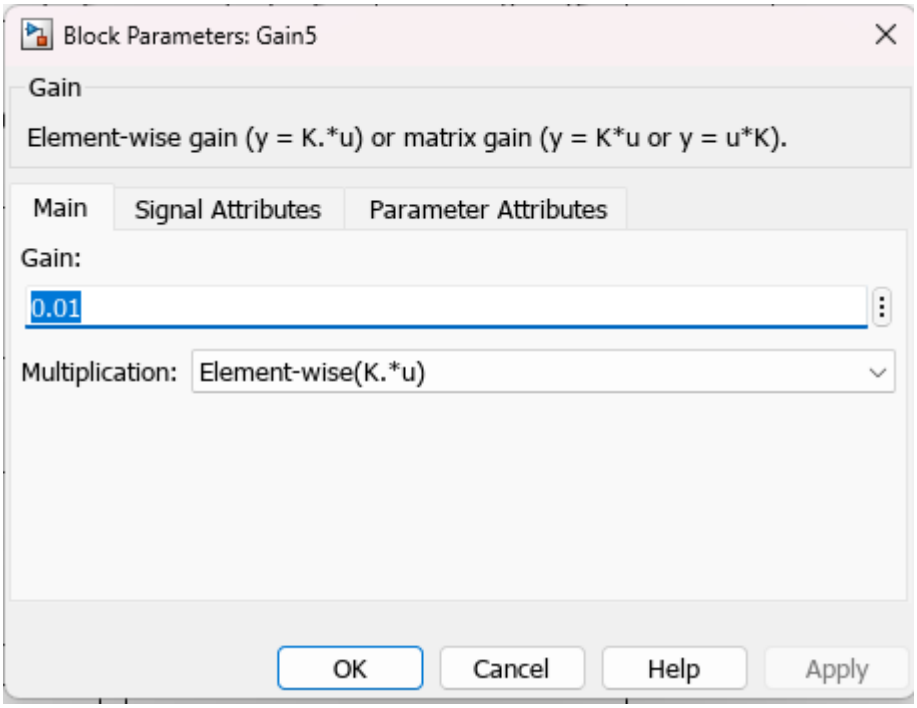

Block Parameters: Gain6
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
4
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

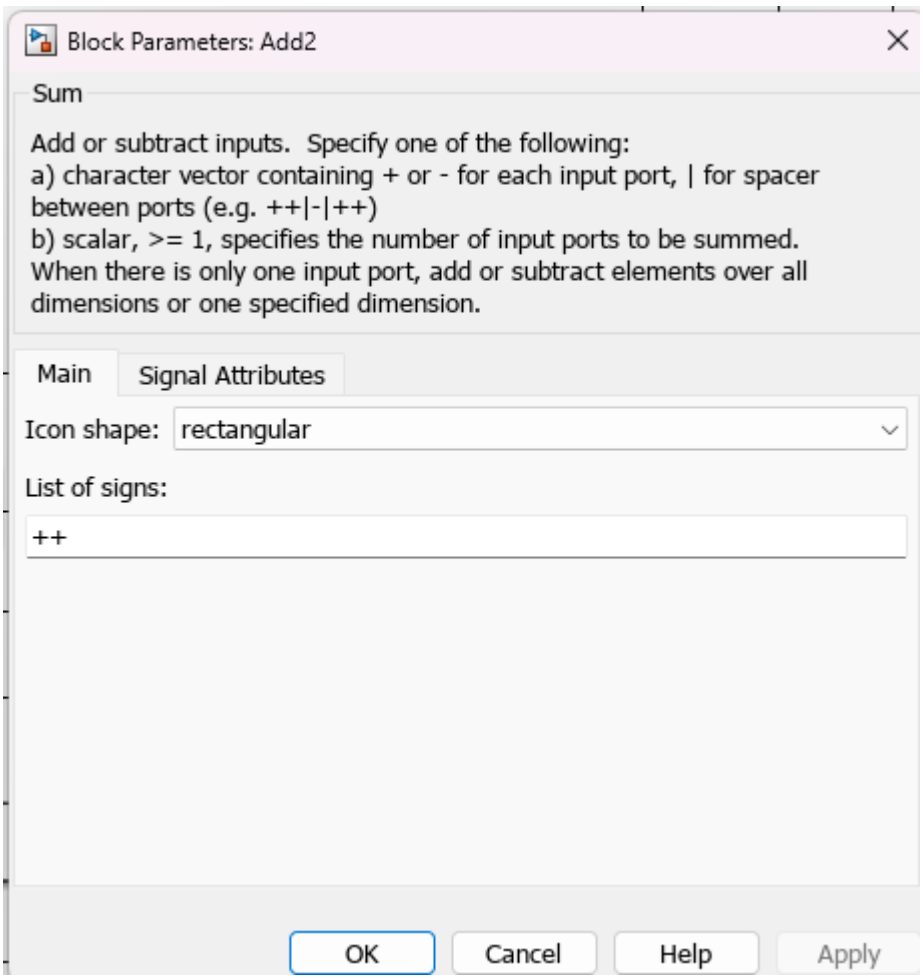

Block Parameters: Gain8
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
1
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

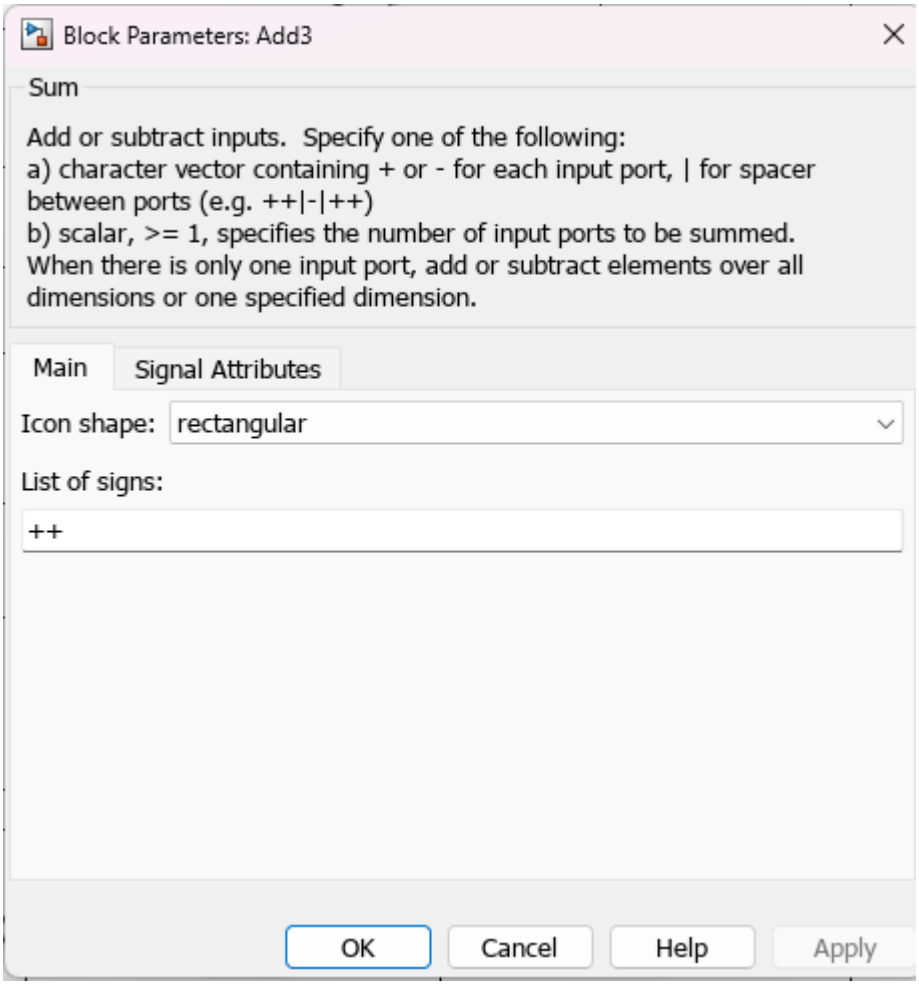

Block Parameters: Gain7
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
-1
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

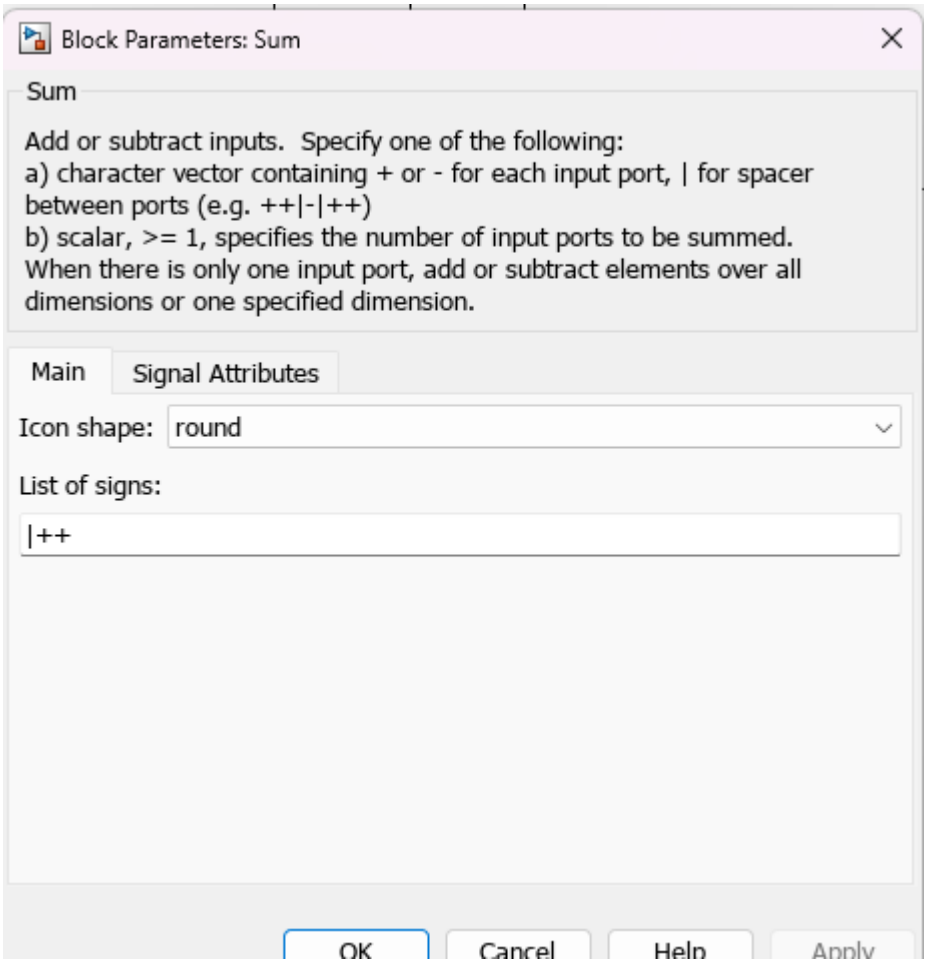

Block Parameters: Gain9
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
2
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

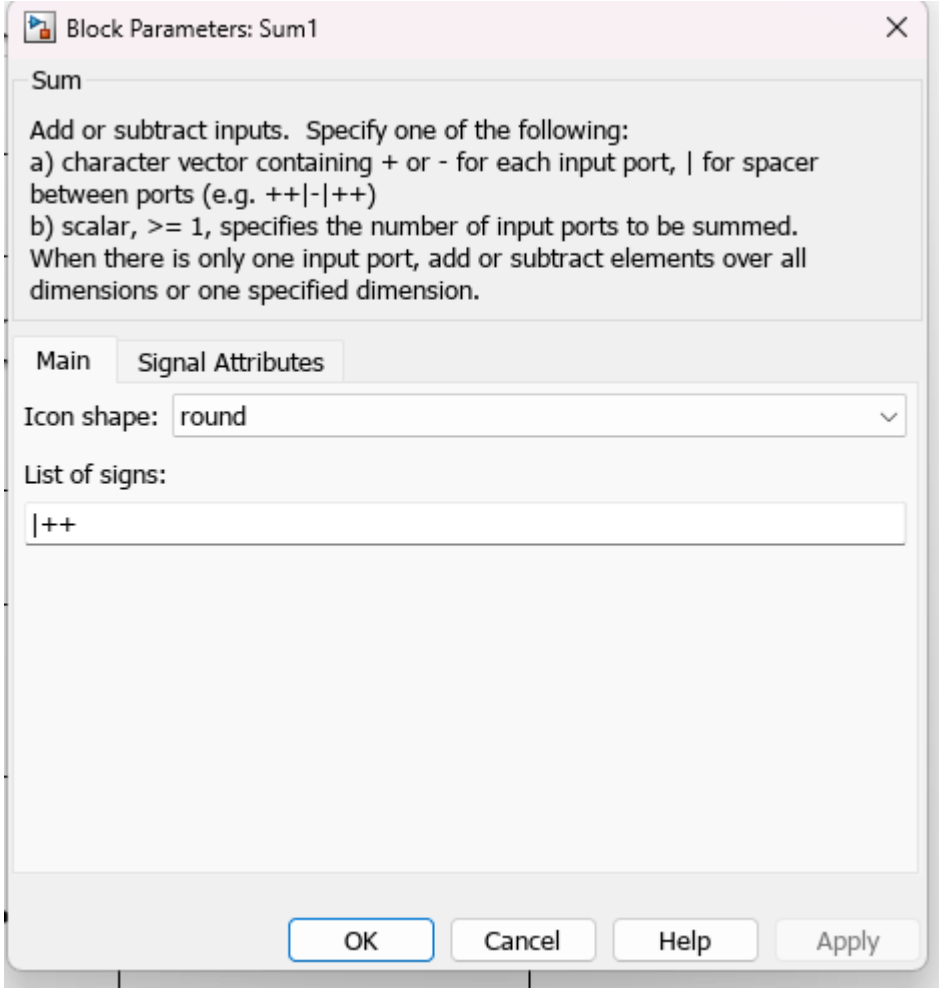

Block Parameters: Gain4
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
0.01
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

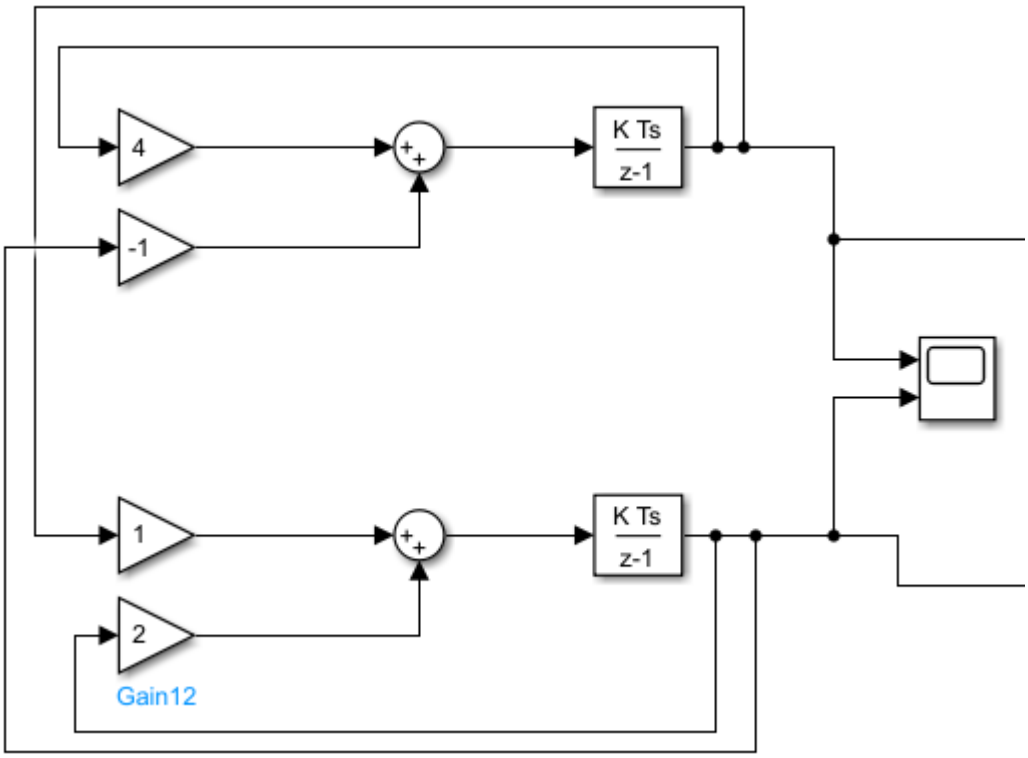

Block Parameters: Gain5
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
0.01
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

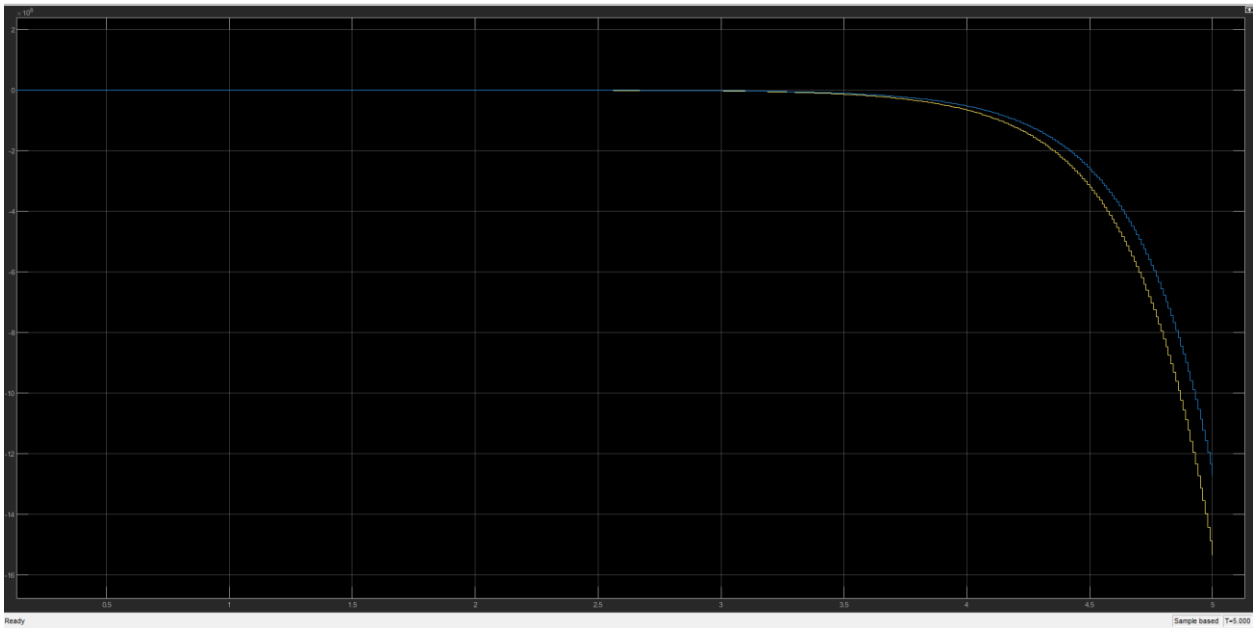

Block Parameters: Add2
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
rectangular
List of signs:
++
OK
Cancel
Help
Apply


Block Parameters: Add3
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
rectangular
List of signs:
++
OK
Cancel
Help
Apply


Block Parameters: Sum
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
round
List of signs:
|++
OK
Cancel
Help
Apply


Block Parameters: Sum1
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
round
List of signs:
|++
OK
Cancel
Help
Apply


4
-1
K Ts
z-1
1
2
Gain12
K Ts
z-1


x 10^6
2
0
-2
-4
-6
-8
-10
-12
-14
-16
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5
Ready
Sample based
T=5.000

Block Parameters: Discrete-Time Integrator

DiscreteIntegrator

Discrete-time integration or accumulation of the input signal.

Main | Signal Attributes | State Attributes

Integrator method: Integration: Forward Euler

Gain value:

1.0

External reset: none

Initial condition source: internal

Initial condition:

-1

Initial condition setting: Auto

Sample time (-1 for inherited):

-1

☐ Limit output

Upper saturation limit:

inf

Lower saturation limit:

-inf

☐ Show saturation port

☐ Show state port

☐ Ignore limit and reset when linearizing

OK | Cancel | Help | Apply

Block Parameters: Discrete-Time Integrator1

DiscreteIntegrator

Discrete-time integration or accumulation of the input signal.

Main | Signal Attributes | State Attributes

Integrator method: Integration: Forward Euler

Gain value:

1.0

External reset: none

Initial condition source: internal

Initial condition:

0

Initial condition setting: Auto

Sample time (-1 for inherited):

-1

☐ Limit output

Upper saturation limit:

inf

Lower saturation limit:

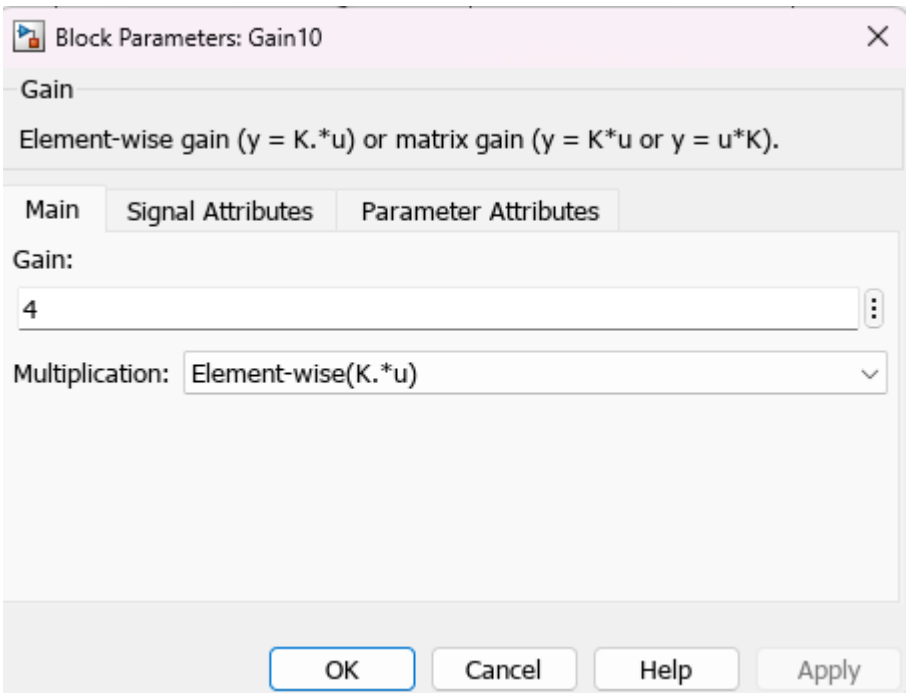
-inf

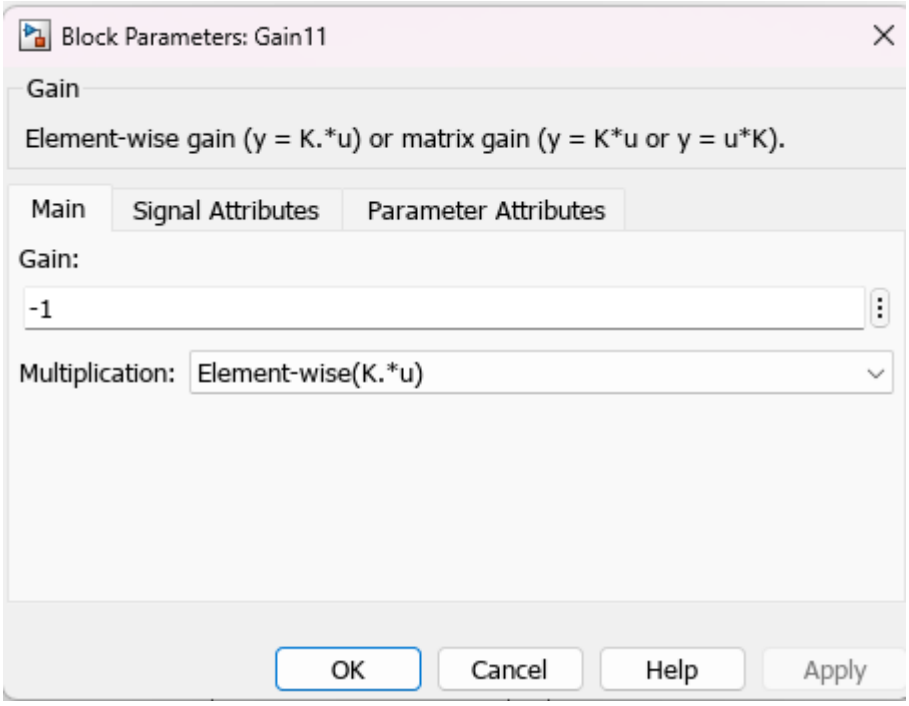
☐ Show saturation port

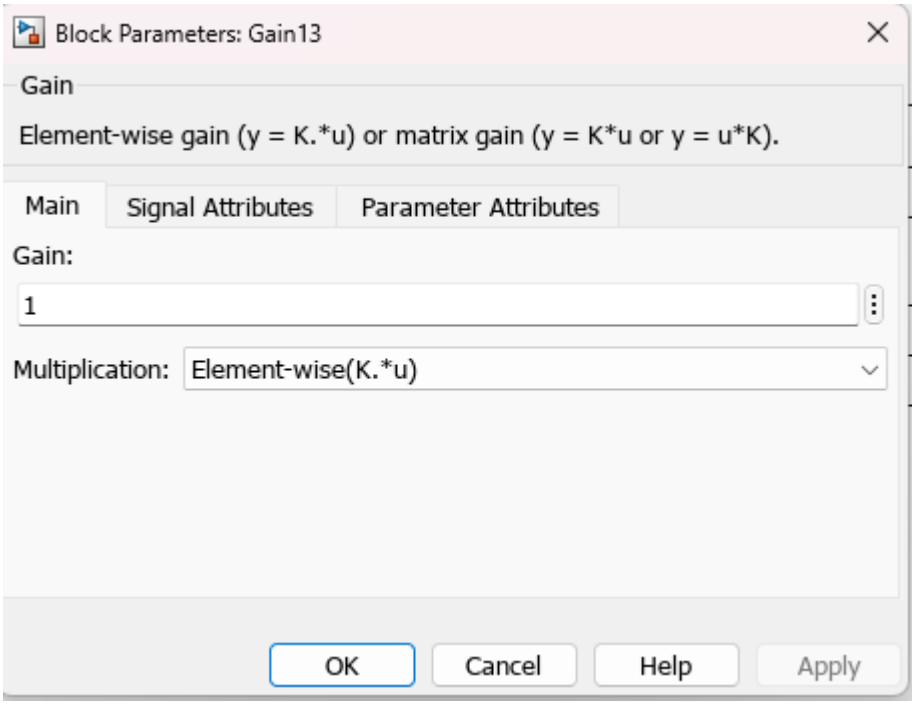
☐ Show state port

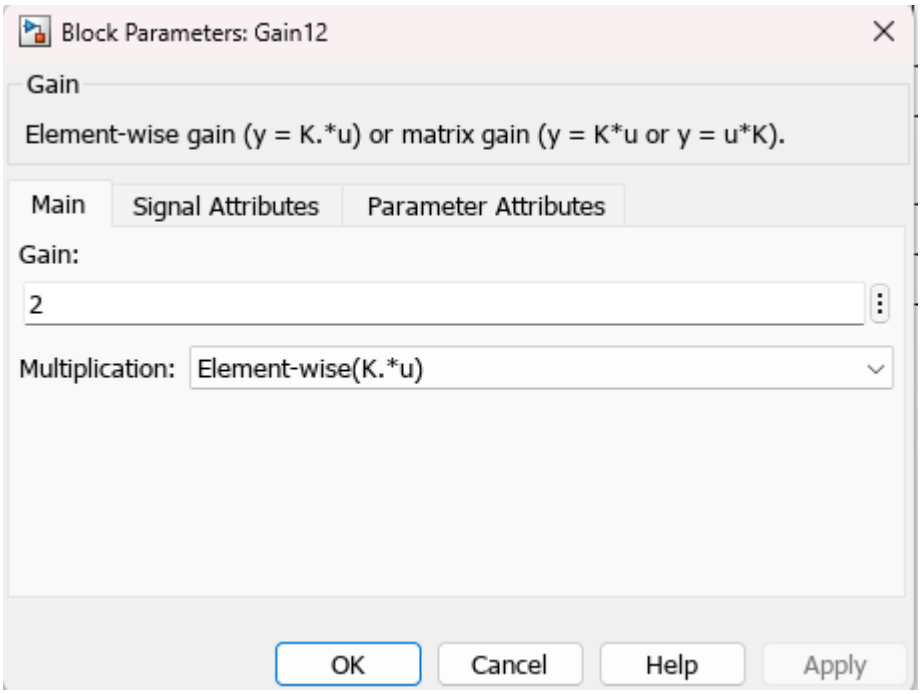
☐ Ignore limit and reset when linearizing

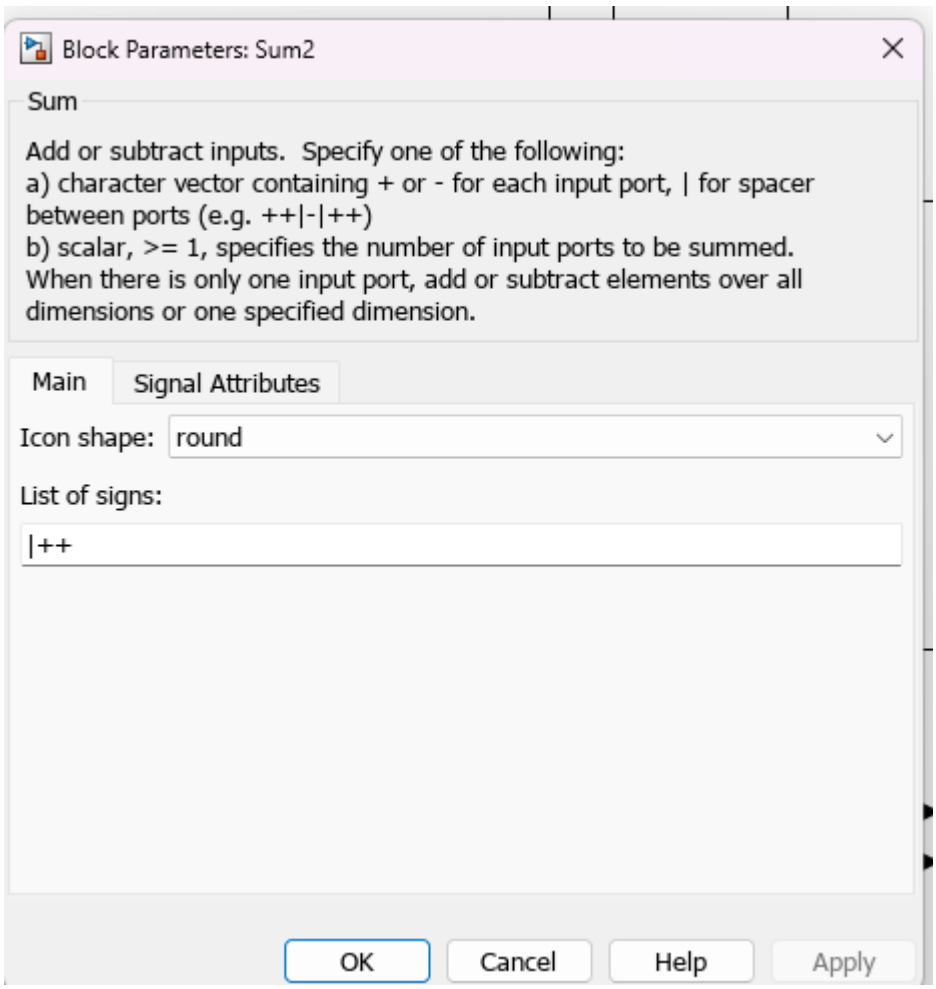
OK | Cancel | Help | Apply

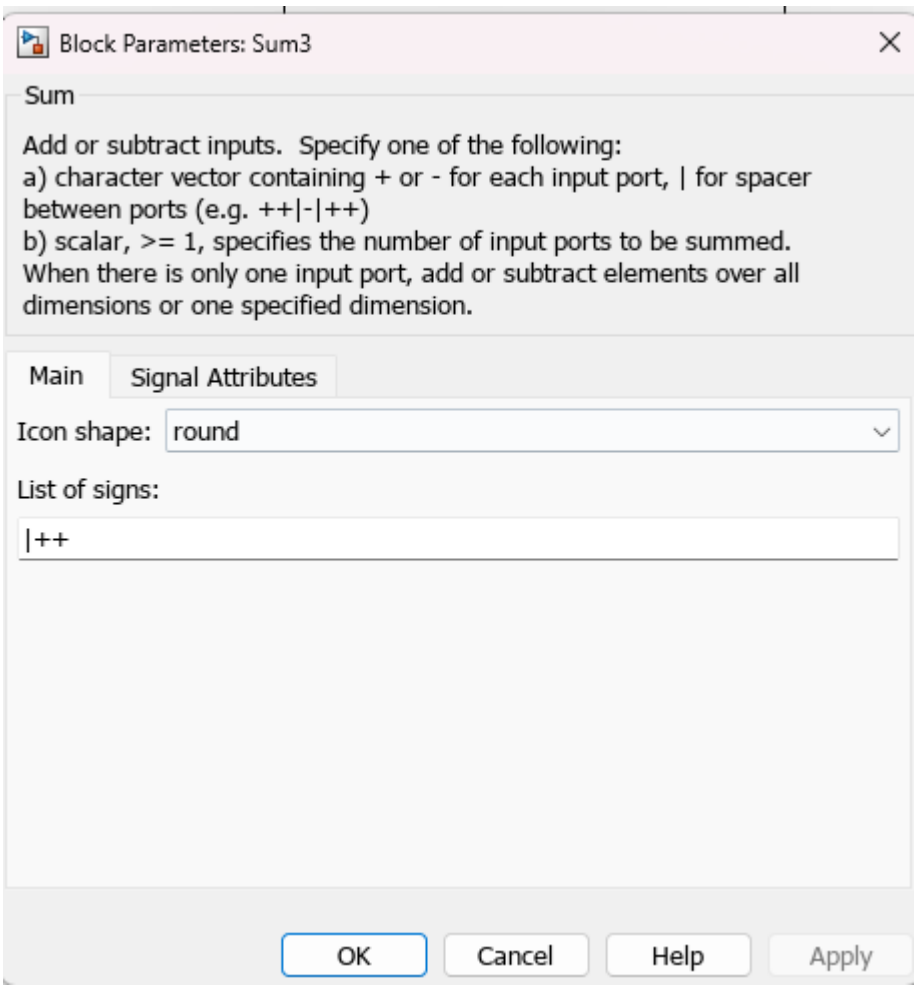

Block Parameters: Gain10
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
4
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

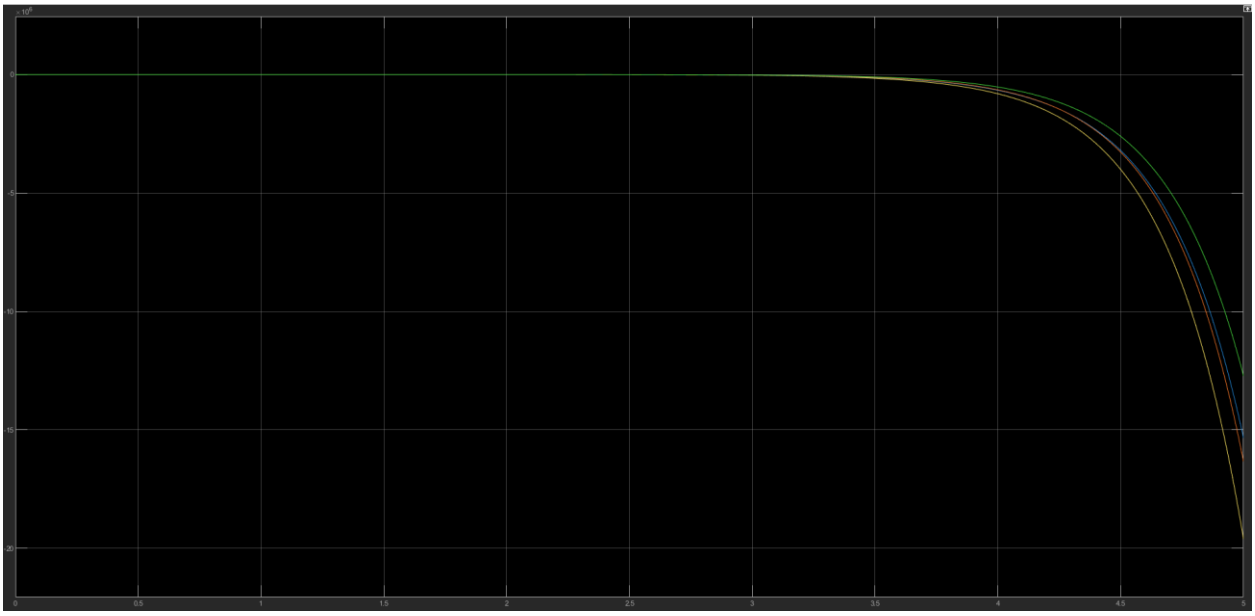

Block Parameters: Gain11
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
-1
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

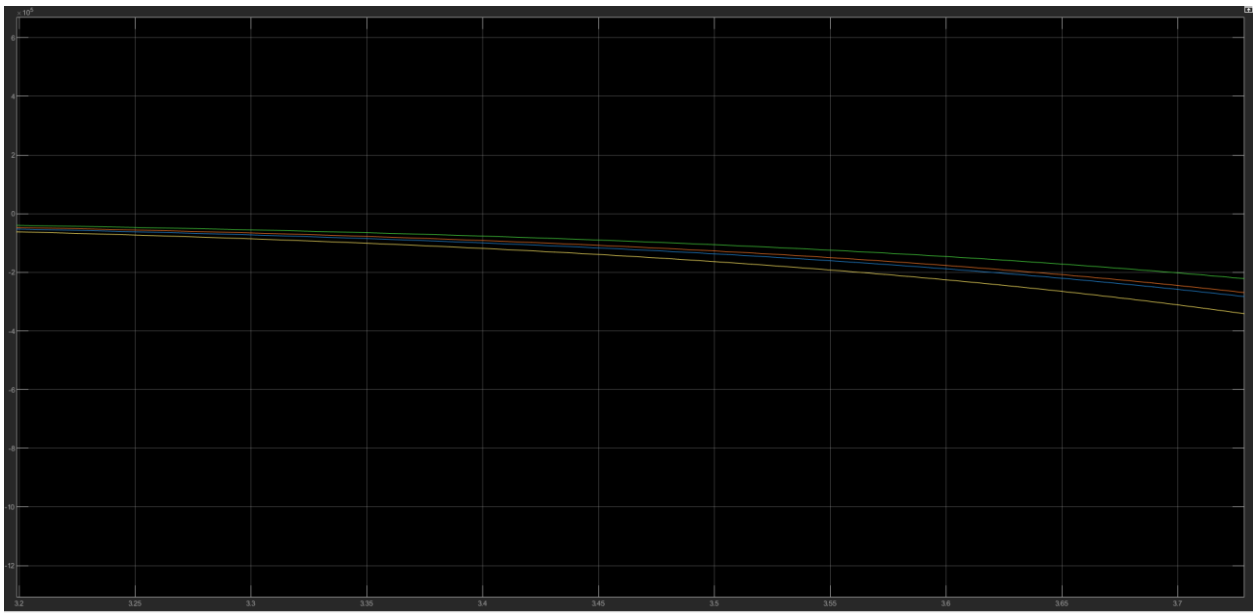

Block Parameters: Gain13
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
1
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply


Block Parameters: Gain12
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
2
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply


Block Parameters: Sum2
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
round
List of signs:
|++
OK
Cancel
Help
Apply


Block Parameters: Sum3
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
round
List of signs:
|++
OK
Cancel
Help
Apply





Вывод:

В ходе выполнения лабораторной работы я успешно применил полученные навыки моделирования к конкретному варианту задач. В первой части работы я решил нелинейное дифференциальное уравнение первого порядка с переменными коэффициентами, построил для него корректную модель в MatLab с использованием решателя ode45, получил численное решение и построил график функции на заданном интервале. Далее я перешёл к системе двух линейных дифференциальных уравнений первого порядка с постоянными коэффициентами и заданными начальными условиями. Для неё я реализовал несколько вариантов моделирования: численное решение через ode45, аналитическое представление через матричную экспоненту, а также построил графики обеих переменных.

Особое внимание было уделено созданию структурных схем в Simulink. Я построил непрерывную модель системы в виде двух интеграторов с обратными связями через блоки Gain и Sum, полностью повторяя классическую реализацию в нормальной форме Коши, при этом все коэффициенты и начальные условия были точно подставлены под мой вариант. Затем я создал дискретные аналоги той же системы: сначала явным методом Эйлера с использованием блоков Unit Delay и Gain, а затем с помощью дискретных интеграторов. Для разных значений шага дискретизации я провёл

сравнение полученных траекторий с точным (непрерывным) решением, наглядно продемонстрировав сходимость дискретной модели при уменьшении шага и рост погрешности при слишком большом шаге.

В результате выполнения всех пунктов я получил полное представление о поведении своей системы: экспоненциальный рост обеих переменных со временем, корректное воспроизведение этого роста как в непрерывных, так и в дискретных моделях при разумных шагах интегрирования. Все построенные модели работали устойчиво, графики были чёткими и информативными, а сравнительный анализ подтвердил теоретические выводы о точности и ограничениях численных методов. Таким образом, поставленные задачи выполнены полностью, навыки работы с MatLab и Simulink существенно закреплены и расширены.